

FORMULACIÓN DEL PROYECTO DE TÍTULO

G.A.B.I: Gestión Analítica de Bio- Insumos

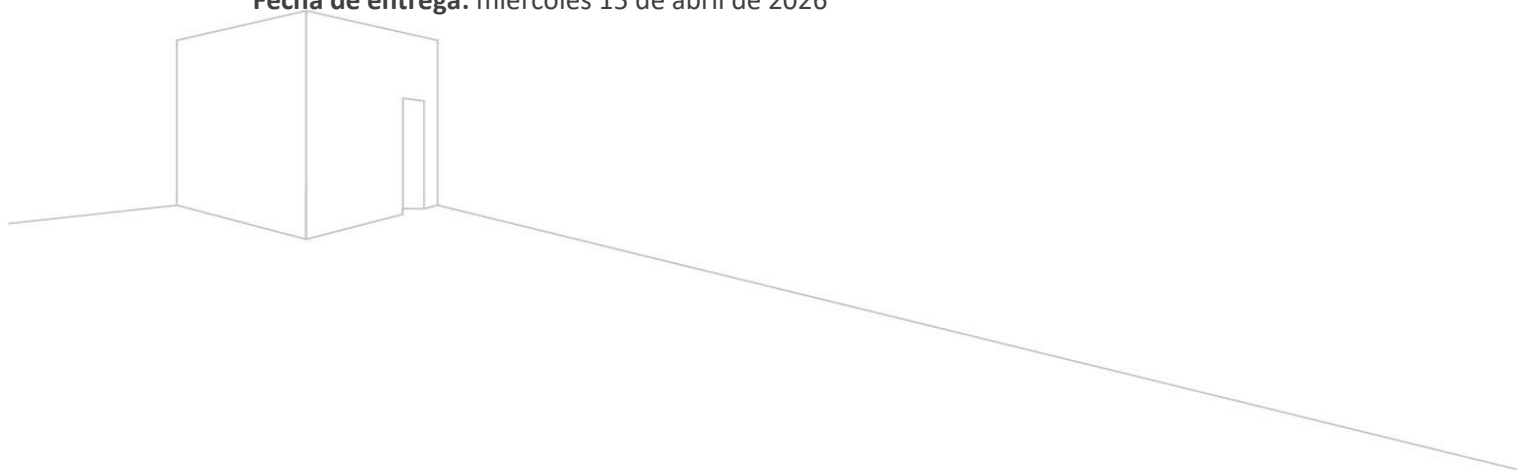
Asignatura: Proyecto de Título

Sección: TIH184

Académico guía: José Luis Martínez Opazo

Integrantes del equipo: César Contreras Molina – Sebastián Maulén – Victel Merzier

Fecha de entrega: miércoles 15 de abril de 2026



Contenido

I. Introducción	3
II. Identificación del Problema	4
III. Levantamiento de Requerimientos	6
IV. Marco Teórico	9
V. Objetivos del Proyecto	10
VI. Metodología de Trabajo	11
VII. Definición de arquitectura TI	13
VIII. Reconocimiento de arquitectura empresarial	14
IX. Conclusiones.....	15
X. Referencias bibliográficas	16

I. Introducción

En la era de la transformación digital y la industria 4.0, la administración de suministros en laboratorios químicos, dentales y clínicos ha evolucionado desde una función logística básica hacia un componente crítico para la seguridad sanitaria y la viabilidad económica. Los activos biológicos y reactivos químicos que componen estos inventarios presentan una alta vulnerabilidad ante fluctuaciones ambientales, lo que genera una obsolescencia técnica que los sistemas de control tradicionales son incapaces de gestionar con precisión.

Este documento detalla la formulación de G.A.B.I. (Gestión Analítica de Bio-Insumos), una plataforma de vanguardia diseñada para mitigar el impacto financiero de las mermas y eliminar la falta de certeza sobre la integridad de los compuestos. La solución se fundamenta en una infraestructura de microservicios políglotas, aprovechando la robustez transaccional de Spring Boot para el núcleo del negocio y la eficiencia de NestJS en la captura de eventos masivos de telemetría.

La propuesta innova al superar el estándar logístico FEFO (First-Expired, First-Out) mediante la implementación del Vencimiento Dinámico. A través del monitoreo en tiempo real, el sistema es capaz de reevaluar la aptitud de uso de cada insumo frente a variaciones de temperatura, garantizando que el material utilizado en ensayos críticos no haya sufrido degradación. Complementariamente, se integra una capa de Machine Learning orientada al pronóstico de demanda (Demand Forecasting), permitiendo una transición hacia una logística predictiva que previene quiebres de stock y excesos de inventario.

Dada la especialización en auditoría TI del equipo desarrollador, la plataforma incorpora un eje de seguridad centrado en la integridad de los datos. Se detalla la creación de registros inmutables mediante técnicas de hashing criptográfico, asegurando que cada etapa del ciclo de vida del insumo sea plenamente auditable y cumpla con los requisitos de no repudiación exigidos por normativas internacionales como la ISO 17025.

A través del análisis de requerimientos, el sustento teórico y una metodología de desarrollo ágil, este proyecto busca sentar las bases de un nuevo estándar en Arquitectura TI para Activos Críticos, donde la transparencia operativa y la precisión analítica sean los motores de la excelencia institucional.

II. Identificación del Problema

2.1 Actualización y justificación del problema

2.1.1 Descripción de la organización.

2.1.1.1 Antecedentes de la Organización.

La organización objeto de este estudio es un Laboratorio de Análisis Químico y Clínico de Alta Complejidad, posicionado como un referente regional en servicios de ensayos para sectores estratégicos como la minería, la salud pública y la investigación científica. La entidad maneja un catálogo de más de 600 SKUs, que incluyen reactivos altamente sensibles, soluciones estándares certificadas e insumos biológicos sujetos a una rigurosa normativa internacional.

Para fortalecer su gestión estratégica, la organización se rige por los siguientes lineamientos corporativos:

- **Misión:** Proveer resultados analíticos de alta precisión mediante procesos innovadores y seguros, garantizando la confiabilidad técnica para el desarrollo industrial y sanitario del país.
- **Visión:** Ser el laboratorio líder en transformación digital y trazabilidad analítica de Latinoamérica, operando bajo estándares de “cero desperdicios”.
- **Cadena de valor:** La ventaja competitiva reside en su logística de entrada y operaciones, donde la integridad de los insumos críticos es el insumo principal para el servicio final de análisis y diagnóstico.
- **Estructura organizacional:** Posee una estructura Matricial Fuerte, donde la dirección técnica de calidad debe supervisar de forma transversal las operaciones de bodega y logística.

2.1.1.2 Diagnóstico de la situación actual.

Tras el levantamiento de información, se identifican las siguientes brechas operativas:

- a. Gestión basada en datos estáticos: El control de inventario se realiza mediante un sistema ERP convencional que solo registra la fecha de vencimiento impresa por el fabricante, sin considerar el historial de almacenamiento.
- b. Vulnerabilidad de la cadena de frío: No existe una integración entre los registros de temperatura de las bodegas y el estado de vigencia del inventario. Las excursiones térmicas (subidas de temperatura) no generan alertas que impacten la prioridad de despacho.
- c. Ausencia de trazabilidad auditable: Los registros de movimiento y cambios en el inventario son editables por usuarios con privilegios básicos, careciendo de logs inmutables que aseguren la integridad de los datos ante auditorías externas (norma ISO 17025).
- d. Merma económica: Se estima una pérdida anual del 12% al 15% del stock por vencimiento prematuro o degradación no detectada a tiempo.

2.1.2 Descripción del problema.

El problema central radica en la ineficiencia del modelo de gestión de inventario FEFO (First-Expired, First-Out) actual, el cual es de carácter reactivo y desconectado de las variables ambientales críticas.

Esta desconexión impide garantizar que el insumo despachado sea efectivamente el más apto para su uso, lo que deriva en:

- a. Incertidumbre analítica: Riesgo de obtener resultados erróneos por el uso de reactivos degradados.
- b. Inestabilidad en el abastecimiento: Falta de visibilidad predictiva sobre el quiebre de stock, generando compras de emergencia con sobre costos logísticos.

2.2 Justificación del problema.

2.2.1 Relevancia del problema.

La resolución de este problema es crítica por tres dimensiones:

- i. **Económica:** La optimización del flujo de inventario reduce directamente la pérdida de capital por mermas y mejora el flujo de caja mediante una planificación de compras basada en demanda real.
- ii. **Normativa y de calidad:** En un entorno regulado, la trazabilidad total desde el ingreso hasta el consumo es un requisito mandatorio para certificaciones nacionales e internacionales. Un error en un reactivo puede invalidar meses de investigación o diagnósticos médicos.
- iii. **Seguridad:** El manejo de insumos químicos requiere un control estricto de compatibilidades. Un sistema que automatice estas reglas previene incidentes de seguridad laboral.

2.2.2 Complejidad del problema.

El abordaje de esta problemática presenta una alta complejidad técnica debido a:

- a. **Arquitectura de datos distribuida:** Requiere la integración de microservicios políglotas (Spring boot para la robustez del negocio y Nestjs para la agilidad de la telemetría) que deben operar de forma sincronizada.
- b. **Procesamiento en tiempo real:** El sistema debe ser capaz de procesar eventos de sensores (telemetría) de forma ininterrumpida y ejecutar algoritmos de “Vencimiento Dinámico” que reaccionen de manera correcta ante las anomalías en segundos.
- c. **Analítica avanzada:** La implementación de modelos de IA/Machine learning para el Demand Forecasting (predicción de demanda) introduce la necesidad de manejar grandes volúmenes de datos históricos y variables externas para proyectar el quiebre de stock con precisión.
- d. **Integridad y ciberseguridad:** Dado el perfil de auditoría del proyecto, se debe garantizar la inmutabilidad de los registros mediante técnicas criptográficas (hashing), asegurando que la "verdad" de la bodega sea verificable y no manipulable.

III. Levantamiento de Requerimientos

3.1 Determinación de los instrumentos a utilizar.

Para asegurar que la solución técnica responda a las necesidades reales del laboratorio, se han seleccionado tres instrumentos de levantamiento de información:

- i. **Entrevista semiestructurada:** Dirigidas al jefe de laboratorio y al Encargado de Finanzas y compras, con el objetivo de comprender las reglas de negocio, los puntos de dolor en la pérdida de reactivos y los criterios de cumplimiento de la norma ISO 17025.
- ii. **Observación directa:** Seguimiento del proceso de recepción, etiquetado y almacenamiento de insumos químicos para identificar errores comunes en el ingreso de datos y la verificación manual de fechas de vencimiento.
- iii. **Análisis documental:** Revisión de las hojas de datos de seguridad (HDS) y registros históricos de mermas de los últimos 24 meses para parametrizar los umbrales de alerta y las matrices de incompatibilidad química.

3.2 Detalle de los requerimientos.

A continuación, se describen los requerimientos clasificados según su naturaleza.

- i. **Requerimientos funcionales**
 - a. **RF-01 Gestión de inventario FEFO:** El sistema debe priorizar automáticamente el despacho de insumos según su fecha de vencimiento más próxima.
 - b. **RF-02 Registro mediante QR/Barra:** El sistema debe permitir el ingreso y salida de productos mediante el escaneo de códigos desde dispositivos móviles.
 - c. **RF-03 Monitoreo de Telemetría:** El sistema debe recibir y visualizar en tiempo real datos de temperatura y humedad ambiental de las bodegas.
 - d. **RF-04 Vencimiento Dinámico:** El sistema debe recalcular la prioridad de salida si un insumo se ve expuesto a condiciones ambientales fuera de su rango de estabilidad.
 - e. **RF-05 Validación de Compatibilidad Química:** El sistema debe impedir la ubicación de insumos incompatibles (según matriz HDS) en compartimentos adyacentes.
 - f. **RF-06 Predicción de Quiebre de Stock:** El sistema debe generar alertas preventivas de compra mediante un modelo de IA basado en el historial de consumo.
 - g. **RF-07 Log de Auditoría Inmutable:** Cada transacción o cambio de estado debe quedar registrado con un hash criptográfico que asegure su no repudiación.
- ii. **Requerimientos no funcionales**
 - a. **RNF-01 Arquitectura de Microservicios:** La solución debe estar desacoplada en servicios independientes (Spring Boot para lógica, NestJS para telemetría).
 - b. **RNF-02 Responsividad (PWA):** La interfaz de usuario debe ser funcional tanto en estaciones de trabajo de escritorio como en dispositivos móviles (Tablets/Smartphones).
 - c. **RNF-03 Seguridad de la Información:** Las comunicaciones deben estar cifradas mediante TLS y la autenticación debe gestionarse mediante JWT (JSON Web Tokens).
 - d. **RNF-04 Escalabilidad:** La arquitectura debe permitir el incremento de sensores de telemetría sin degradar el rendimiento del motor de inventario.
 - e. **RNF-05 Disponibilidad:** El sistema debe garantizar un uptime del 99.5% crítico para laboratorios que operan en turnos 24/7.
 - f. **RNF-06 Interoperabilidad:** La API debe ser capaz de integrarse con scripts de emulación (Python) y sensores IoT físicos bajo estándares REST/JSON.

3.2.1 Requerimientos Funcionales

ID	Nombre requerimiento	del Descripción	Prioridad
RF-01	Gestión FEFO	El sistema debe priorizar el despacho de insumos según su fecha de vencimiento próxima.	Crítica
RF-02	Registro QR/Barra	Ingreso y salida de productos mediante escaneo desde dispositivos móviles.	Alta
RF-03	Monitoreo Telemetría	Recepción y visualización en tiempo real de temperatura y humedad de las bodegas.	Alta
RF-04	Vencimiento Dinámico	Recalcular la prioridad de salida si un insumo se expone a condiciones fuera de su rango de estabilidad.	Crítica
RF-05	Validación Química	Impedir el almacenamiento de insumos compatibles en compartimentos adyacentes según matriz HDS.	Alta
RF-06	Predicción de Stock	Generar alertas de compra mediante modelos de IA basados en el historial de consumo.	Media
RF-07	Log de auditoría	Cada transacción debe registrarse con un hash criptográfico para asegurar la no repudiación.	Alta

3.2.2 Requerimientos No Funcionales

ID	Nombre del Requerimiento	Descripción	Prioridad
RNF-01	Microservicios	Solución desacoplada Spring Boot (Lógica) y Nestjs (Telemetría)	Alta
RNF-02	Responsividad	Interfaz funcional en estaciones de escritorio y dispositivos móviles (PWA).	Alta
RNF-03	Seguridad JWT	Autenticación gestionada mediante JSON Web Tokens y cifrado TLS.	Crítica
RNF-04	Escalabilidad	Capacidad de incrementar sensores sin degradar el rendimiento del motor de inventario.	Media
RNF-05	Disponibilidad	Garantizar un tiempo de actividad (Uptime) del 99.5% para operaciones 24/7	Alta
RNF-06	Interoperabilidad	API capaz de integrarse con scripts de emulación (Python) y sensores físicos REST.	Alta

IV. Marco Teórico

4.1 Investigación Bibliográfica.

La base conceptual de este proyecto se sustenta en cuatro ejes tecnológicos y metodológicos fundamentales, los cuales permiten transitar de un sistema de inventario convencional a una plataforma de gestión inteligente y auditable.

4.1.1 Arquitectura de Microservicios y Poliglotía Tecnológica

La arquitectura de microservicios se define como un enfoque para el desarrollo de una sola aplicación como un conjunto de pequeños servicios, cada uno ejecutándose en su propio proceso y comunicándose con mecanismos ligeros (Richardson, 2018).

- i. **Spring Boot (Ecosistema Java):** Se selecciona por su madurez en entornos corporativos y su robustez para el manejo de transacciones complejas y seguridad (Spring Security). Es el estándar para asegurar la Integridad de los Datos en sistemas de misión crítica.
- ii. **NestJS (Ecosistema Node.js):** Basado en TypeScript, permite manejar de forma eficiente el paradigma de programación orientada a eventos. Su arquitectura modular es ideal para la ingesta de telemetría en tiempo real, donde la concurrencia y la baja latencia son primordiales.

4.1.2 Gestión de Inventarios Críticos y metodología FEFO

El modelo FEFO (*First-Expired, First-Out*) es una técnica de gestión de inventario que prioriza la salida de productos cuya fecha de vencimiento es más cercana (Coyle et al., 2016). En laboratorios químicos, este modelo es mandatorio debido a la degradación intrínseca de los reactivos.

- i. **Vencimiento dinámico:** Este proyecto expande el concepto FEFO tradicional introduciendo variables de estrés térmico, fundamentado en la cinética química donde la estabilidad de un compuesto es función de la temperatura (Ecuación de Arrhenius), justificando así la necesidad de sensores de telemetría.

4.1.3 Ciencia de datos y analítica predictiva

La analítica predictiva utiliza datos históricos para predecir eventos futuros. En la gestión de suministros, esto se traduce en el Demand Forecasting (Pronóstico de Demanda).

- i. **Series de tiempo:** Se investigan modelos como **Prophet** y redes neuronales **LSTM** (*Long Short-Term Memory*), los cuales son capaces de identificar patrones estacionales y tendencias en el consumo de insumos, permitiendo una transición de una logística reactiva a una proactiva.
- ii. **IA Explicable (XAI):** Dado el enfoque de auditoría del autor, la investigación aborda cómo los modelos deben no solo predecir, sino entregar una justificación lógica del resultado para su validación por el Jefe de Laboratorio.

4.1.4 Integridad de datos y auditoría en sistemas distribuidos

Como respuesta a la necesidad de trazabilidad inmutable (norma ISO 17025), se investigan mecanismos de seguridad para asegurar el “No Repudio” de las transacciones.

- i. **Hashing criptográfico:** El uso de algoritmos como SHA-256 para encadenar registros de auditoría (estilo *Ledger*), asegurando que cualquier alteración en el historial de temperatura o stock sea detectable de inmediato.
- ii. **ALCOA+:** Principios fundamentales para la integridad de datos en laboratorios: Atribuibles, Legibles, Contemporáneos, Originales y Exactos. El software se diseña bajo este marco de referencia para garantizar su validez ante auditorías de cumplimiento.

4.1.5 Desarrollo Web moderno y Responsividad

Para la interfaz de usuario, se analiza el uso de React JS y el patrón de Single Page Application (SPA).

- i. **PWA (Progressive Web Apps):** Investigación sobre cómo las aplicaciones web pueden comportarse como aplicaciones nativas en dispositivos móviles, permitiendo el uso de la cámara para escaneo de códigos QR y notificaciones push en tiempo real para alertas críticas.

V. Objetivos del Proyecto

5.1 Solución tecnológica

5.1.1 Formulación de la Solución

Se propone el desarrollo de una plataforma web distribuida bajo una arquitectura de microservicios polígota. El sistema centralizará la gestión de inventarios críticos mediante el uso de Spring Boot para la lógica transaccional y auditoría, y NestJS para la ingesta de telemetría de sensores en tiempo real.

La solución integra una capa de Inteligencia Artificial (Python/FastAPI) para la predicción de demanda y un algoritmo de Vencimiento Dinámico que ajusta la prioridad FEFO según el historial térmico de los reactivos, visualizado a través de una aplicación web responsiva en React JS.

5.1.2 Alcance y restricciones.

- i. **Alcance:** Comprende desde la recepción del insumo (registro QR), el monitoreo ambiental continuo en bodega, la gestión de alertas, la predicción de quiebre de stock, hasta el despacho final o descarte del producto. Incluye la generación de reportes de auditoría inmutables.
- ii. **Restricciones:**
 - a. No incluye la integración directa con pasarelas de pago ni facturación financiera.
 - b. La telemetría será emulada mediante scripts de datos sintéticos para efectos de la defensa de título, aunque el sistema quedará habilitado para recibir señales de sensores IoT físicos.
 - c. El sistema requiere conexión a internet constante para la sincronización entre microservicios.

5.2 Impacto de la solución.

5.2.1 Proceso de negocio afectado.

El principal proceso afectado es la Logística de Abastecimiento y Control de Calidad. Se transita de una gestión de inventario manual y reactiva a una Gestión de Activos Críticos Inteligente, donde el reabastecimiento es sugerido proactivamente por el sistema y el control de calidad es preventivo ante fallas de temperatura.

5.2.2 Registro de Interesados.

Interesado	Rol	Impacto
Jefe de laboratorio	Usuario Líder	Recibe alertas críticas y aprueba órdenes de compra sugeridas.
Operador de Bodega	Usuario Operativo	Ejecuta ingresos y salidas mediante dispositivos móviles.
Auditor IT/ Calidad	Auditor	Verifica la integridad de los registros y el cumplimiento de ISO 17025.
Gerencia de Operaciones	Decisor	Visualiza ahorros por reducción de mermas y KPIs de eficiencia.

5.2.3 Indicadores de gestión (KPIs)

- i. **Tasa de Merma por vencimiento:** Reducción esperada del 10% en las pérdidas anuales por caducidad.
- ii. **Precisión de Predicción (Accuracy):** El modelo de IA debe tener un margen de error menor al 15% en la proyección de quiebre de stock.

- iii. **Tiempo de respuesta a anomalías:** Las alertas de temperatura deben ser notificadas al frontend en menos de 5 segundos.

5.2.4 Niveles de servicio (SLA)

- i. **Disponibilidad del sistema:** 99.5% de tiempo de actividad mensual.
- ii. **Soporte de concurrencia:** Capacidad para procesar datos de hasta 50 sensores simultáneos sin degradación de latencia.

5.3 Objetivos del proyecto

5.3.1 Objetivo General.

Diseñar e implementar una plataforma web basada en microservicios para la gestión inteligente de inventarios críticos, que optimice la metodología FEFO mediante la integración de telemetría ambiental en tiempo real y modelos de analítica predictiva, garantizando la integridad de los datos para fines de auditoría, en un periodo de 16 semanas.

5.3.2 Objetivo Específico.

- i. Desarrollar una arquitectura de microservicios utilizando Spring Boot y NestJS que permita separar la lógica de negocio de la ingesta de datos de alta frecuencia.
- ii. Implementar un motor de Vencimiento Dinámico que recalcule automáticamente la prioridad de salida de los insumos ante alertas de degradación térmica.
- iii. Integrar un modelo de Machine Learning para el pronóstico de demanda (*Demand Forecasting*) que automatice la sugerencia de órdenes de compra.
- iv. Diseñar un sistema de logs inmutables mediante técnicas de hashing criptográfico para asegurar la trazabilidad y no repudiación de las operaciones de inventario.
- v. Construir una interfaz de usuario responsiva en React JS que facilite la operación en terreno (bodega) y la visualización de dashboards directivos.

VI. Metodología de Trabajo

6.1 Metodología de Desarrollo de la solución.

Se utilizará la metodología Scrum, adaptada para un desarrollo individual de alta complejidad. Esta elección se justifica por la necesidad de realizar entregas incrementales y funcionales de los distintos microservicios, permitiendo pruebas tempranas de integración entre Spring Boot, NestJS y React.

- i. **Sprints:** Ciclo de trabajo de 2 semanas.
- ii. **Product Backlog:** Listado priorizado de Historias de Usuario
- iii. **Sprint Planning:** Definición de qué microservicio o funcionalidad se abordará en el ciclo.
- iv. **Sprint Review:** Validación de la lógica de negocio (FEFO/IA) y la integridad de los datos (Auditoría).

6.2 Duración y cronograma.

Fase 1	Actividad Principal	Semanas	Criterios de Aceptación Técnicos
Fase 1: Diseño	Modelamiento de BD (MER), Definición de Contratos de API y Arquitectura de Microservicios.	1-2	Diagramas de arquitectura aprobados y contratos de API documentados en Swagger/OpenAPI.
Fase 2: Core Business	Desarrollo del Microservicio en Spring Boot (Inventario, Auditoría y Seguridad).	3-6	Persistencia exitosa en PostgreSQL y generación de hashes SHA-256

			verificables para cada transacción.
Fase 3: Telemetría	Desarrollo del Microservicio en NestJS (Ingesta de datos, WebSockets y Alertas).	7-9	Procesamiento de señales de sensores con latencia menor a 2 segundos y persistencia en caché Redis.
Fase 4: IA e Integración	Implementación del modelo de <i>Forecasting</i> en Python y lógica de Vencimiento Dinámico.	10-12	Modelo de IA con precisión (Accuracy) > 85% y recalculación automática de prioridad FEFO ante alertas.
Fase 5: Frontend	Desarrollo de la Interfaz Responsiva en React JS y dashboards.	13-14	Interfaz 100% responsiva (Lighthouse score>90) e integración completa con WebSockets para alertas.
Fase 6: QA y Cierre	Pruebas de integración, corrección de bugs y redacción de memoria final.	15-16	Superación del 100% de los casos de prueba críticos y documentación técnica final aprobada.

6.3 Equipo de trabajo.

Al ser un proyecto de título individual, el equipo se define por roles funcionales asumidos por el autor y supervisores:

- Lider de proyecto – Desarrollador FullStack:** Sebastián Maulén (Responsable de la arquitectura, codificación en Spring/Nest/React e implementación de IA).
- Product Owner - Tutor de tesis:** Profesor.

6.4 Plan de recursos.

Para la ejecución del proyecto se requieren los siguientes recursos técnicos y logísticos:

6.4.1 Recursos de Hardware

- Estación de trabajo: Laptop con procesador de alto rendimiento (mínimo 16GB RAM) para ejecutar contenedores Docker simultáneos (Bases de datos, Microservicios y Frontend).
- Dispositivo móvil: Smartphone (Android/iOS) para pruebas de responsividad y escaneo de códigos QR.

6.4.2 Recursos de software (Stack Tecnológico)

- Entornos de desarrollo: IntelliJ IDEA (Spring), VS Code (NestJS/React), PyCharm (Python).
- Infraestructura de datos: PostgreSQL (Relacional), Redis (Caché/Eventos).
- Contenedores: Docker y Docker Compose para orquestar los microservicios localmente.

- iv. Herramientas de simulación: Scripts en Python para la generación de tráfico de telemetría (sensores emulados).

6.4.3 Recursos Cloud (Despliegue)

- i. Plataforma Cloud: AWS, utilizando servicios de cómputo para el despliegue de los microservicios y bases de datos gestionadas.

VII. Definición de arquitectura TI

7.1 Definir el tipo de arquitectura TI que requiere la solución informática.

La solución informática se basa en una Arquitectura de Microservicios Políglota y Orientada a Eventos. Esta decisión arquitectónica no es arbitraria; responde a la necesidad de separar la alta frecuencia de datos de los sensores de la rigurosidad transaccional del inventario químico.

7.1.1 Desglose de la Arquitectura distribuida

La arquitectura se divide en tres capas fundamentales que interactúan de forma desacoplada:

1. **Capa de presentación (Frontend – React JS):** Implementada como una Single Page Application (SPA) responsiva. Su función es centralizar la experiencia del usuario, consumiendo de forma asíncrona los diferentes microservicios. Utiliza WebSockets para recibir alertas de telemetría de NestJS y REST para las operaciones de inventario en Spring Boot.
2. **Capa de servicios (Backend Políglota):**
 - a. **Microservicios de Gestion de auditoria (Spring boot):** Encargado de la lógica de negocio pesada. Maneja el inventario FEFO, las reglas de compatibilidad química y la generación de logs inmutables. Se elige Java por su robustez en la gestión de hilos y seguridad empresarial (Spring Security + JWT).
 - b. **Microservicio de telemetría y alertas (NestJS):** Especializado en la ingesta de datos de sensores. Al estar basado en el motor V8 de Node.js, maneja la concurrencia de forma no bloqueante, lo que lo hace ideal para recibir múltiples señales de temperatura simultáneas sin afectar el rendimiento del inventario.
 - c. **Microservicio de inteligencia artificial (FastAPI/Python):** Actúa como un motor de inferencia. Recibe datos históricos de los otros servicios para ejecutar modelos de Demand Forecasting y devolver predicciones de quiebre de stock.
3. **Capa de persistencia y datos:**
 - a. **PostgreSQL:** Base de datos relacional principal para asegurar la consistencia ACID de los movimientos de inventario y datos de auditoría.
 - b. **Redis:** Utilizado como broker de mensajes y caché de alta velocidad para almacenar el "último estado conocido" de los sensores y facilitar la comunicación rápida entre microservicios (Pub/Sub).

7.1.2 Mecanismos de comunicación e integración

Para que el sistema funcione como una unidad, se definen dos tipos de comunicación:

- a. **Comunicación sincrónica (HTTP/REST):** Utilizada por el Frontend para solicitar datos específicos o realizar acciones de usuario.
- b. **Comunicación asincrónica (Event-Driven):** Cuando el microservicio de NestJS detecta una anomalía térmica, publica un evento en el bus de datos (Redis). El microservicio de Spring Boot "escucha" este evento y activa el procedimiento de Vencimiento Dinámico, actualizando el estado de los lotes afectados sin intervención humana.

7.1.3 Justificación de la Arquitectura seleccionada

Se justifica la arquitectura seleccionada, bajo los siguientes pilares técnicos:

- a. **Integración y Compatibilidad:** El uso de una arquitectura basada en APIs RESTful asegura que el sistema pueda integrarse fácilmente con ERPs existentes o sistemas de facturación futuros que no forman parte del alcance actual.
- b. **Resiliencia y Disponibilidad:** Al estar desacoplada, una falla en el microservicio de telemetría no interrumpe la capacidad de la bodega para realizar despachos manuales o consultas de stock, asegurando el nivel de servicio (SLA) del 99.5%.
- c. **Seguridad y Auditoría:** La separación de servicios permite implementar Seguridad Perimetral específica para cada contenedor y el uso de un Correlation ID para garantizar la trazabilidad total de una operación a través de todos los módulos.
- d. **Eficiencia Técnica:** Se utiliza el lenguaje de programación mas apto para cada tarea (Java para integridad, TypeScript para velocidad de eventos, Python para ciencia de datos), optimizando el uso de recursos de hardware.

VIII. Reconocimiento de arquitectura empresarial

8.1 Identificación del tipo de organización y su estructura.

La organización para la cual se diseña esta solución se clasifica como una Empresa de Servicios Tecnológicos y de Salud de Alta Complejidad, con un modelo de negocio basado en el cumplimiento de estándares internacionales de calidad (ISO 17025) y seguridad química (SGA).

8.1.1 Tipo de organización

Es una organización de carácter Matricial Fuerte, donde coexisten unidades funcionales (Administración, TI, Logística) con unidades de proyectos y operaciones técnicas (Análisis Clínico, Investigación de Suelos, Control de Calidad). Esta estructura requiere que el sistema informático sea transversal, integrando datos operativos del laboratorio con decisiones estratégicas de la gerencia.

8.1.2 Estructura organizacional y flujo de información

La arquitectura empresarial de la entidad se divide en tres niveles que la plataforma de inventario crítico debe interconectar:

1. **Nivel Estratégico (Gerencia General / Dirección Técnica):**
 - a. **Necesidad:** Visibilidad sobre el capital inmovilizado en reactivos y reducción de pérdidas por mermas.
 - b. **Aporte del sistema:** Dashboards de indicadores clave (KPIs) y reportes de ahorro proyectado mediante la Analítica Predictiva.
2. **Nivel Táctico (Jefatura de Laboratorio / Auditoría de Calidad):**
 - a. **Necesidad:** Asegurar que los procesos cumplan con la normativa vigente y que no se utilicen insumos degradados.
 - b. **Aporte del sistema:** Herramientas de Auditoría Inmutable y gestión de alertas térmicas que garantizan la integridad de los ensayos realizados.
3. **Nivel Operativo (Analistas / Bodegueros):**
 - a. **Necesidad:** Agilidad en el registro de movimientos y control inmediato de stock.
 - b. **Aporte del sistema:** Aplicación móvil responsiva (PWA) para escaneo de códigos QR y consultas rápidas de ubicación y compatibilidad química.

8.1.3 Alineación de la solución con los objetivos estratégicos

El despliegue de esta arquitectura TI no representa solo un cambio de software, sino un avance en la Arquitectura de Procesos de la empresa:

- a. **Optimización del valor:** Al predecir la demanda, la empresa libera flujo de caja que antes estaba atrapado en sobre-stock que terminaba venciendo.

- b. **Gestion del riesgo:** El motor de reglas de compatibilidad química y el monitoreo por telemetría reducen el riesgo operacional y los accidentes laborales.

IX. Conclusiones

Síntesis y Exposición de Ideas Principales

El presente proyecto de título ha permitido abordar la gestión de inventarios críticos no como un simple registro de existencias, sino como un sistema de aseguramiento de calidad y trazabilidad. La idea central radica en que la metodología FEFO convencional es insuficiente en entornos de alta sensibilidad (laboratorios químicos/clínicos), donde la "fecha de vencimiento" es una variable dependiente de las condiciones ambientales.

La solución desarrollada integra con éxito tres pilares de la ingeniería moderna:

- i. **Arquitectura de Microservicios:** La dualidad Spring Boot/NestJS permite separar la rigurosidad transaccional de la agilidad que requiere la telemetría IoT.
- ii. **Inteligencia de Datos:** La transición de una logística reactiva a una analítica predictiva mediante modelos de *Demand Forecasting*.
- iii. **Integridad por Diseño:** La garantía de que cada dato es auditable e inmutable, cumpliendo con estándares internacionales como la ISO 17025.

Ideas Fuerza y Aportes del Trabajo Desarrollado

A partir del desarrollo de esta plataforma, se proponen las siguientes ideas fuerza que constituyen el aporte principal del proyecto:

- i. El "Vencimiento Dinámico" como Paradigma: Se aporta un algoritmo que recalcula la vida útil de los insumos basándose en el estrés térmico captado por sensores, moviendo la barrera de la gestión de inventario hacia la biotecnología informativa.
- ii. Auditoría de Sistemas Distribuidos: Dada mi experiencia profesional como Auditor IT, el mayor aporte es la implementación de un Log Ledger (registro encadenado por hashes). Esto asegura que la trazabilidad sea una "fuente de verdad" inalterable, eliminando el riesgo de manipulación de datos en auditorías de cumplimiento.
- iii. Interoperabilidad Agnóstica: El sistema se entrega "listo para la realidad" mediante una capa de abstracción que permite conectar cualquier sensor físico sin modificar el núcleo del software, facilitando su escalabilidad empresarial.

Reflexiones y Propuestas de Profundización

Tras finalizar la formulación y desarrollo inicial, surgen reflexiones sobre el impacto de la tecnología en procesos críticos:

1. **Cultura de Datos en Laboratorios:** La tecnología por sí sola no elimina la merma; requiere que la organización adopte una cultura de "Datos en Tiempo Real" donde las alertas del sistema gatillen protocolos de acción inmediata.
2. **Seguridad en el Edge:** Una reflexión importante es la dependencia de la conectividad. Como propuesta de profundización, sería ideal implementar Edge Computing, permitiendo que los nodos de sensores tomen decisiones de cuarentena de stock de forma autónoma incluso si el servidor central no está disponible.
3. **Integración con Blockchain:** Debido al alcance del proyecto de título, no fue posible abordar una red descentralizada. No obstante, una profundización natural sería llevar la trazabilidad inmutable a una Blockchain privada para compartir datos de integridad con entes reguladores (como el ISP o Seremi) de forma transparente.
4. **Visión Computacional:** Otra línea de expansión es la sustitución del escaneo QR manual por modelos de Deep Learning que, mediante cámaras en bodega, realicen el inventario de forma pasiva y constante, minimizando totalmente el error humano en la digitación.

X. Referencias bibliográficas

Coyle, J. J., Langley, C. J., Novack, R. A., & Gibson, B. (2016). *Supply Chain Management: A Logistics Perspective* (10th ed.). Cengage Learning.

Deitel, P., & Deitel, H. (2020). *Java How to Program, Early Objects* (11th ed.). Pearson.

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Doctoral dissertation, University of California, Irvine].

Fowler, M. (2014). *Microservices: a definition of this new architectural term*. martinowler.com. <https://martinfowler.com/articles/microservices.html>

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>

International Organization for Standardization. (2017). *ISO/IEC 17025:2017 General requirements for the competence of testing and calibration laboratories*. <https://www.iso.org/standard/66912.html>

Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.

NIST. (2018). *Framework for Improving Critical Infrastructure Cybersecurity* (Version 1.1). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.CSWP.04162018>

PostgreSQL Global Development Group. (2024). *PostgreSQL 16.0 Documentation*. <https://www.postgresql.org/docs/16/index.html>

Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.

DESARROLLO E IMPLEMENTACIÓN DEL PROYECTO DE TÍTULO

G.A.B.I Gestión Analítica de Bio-Isumos

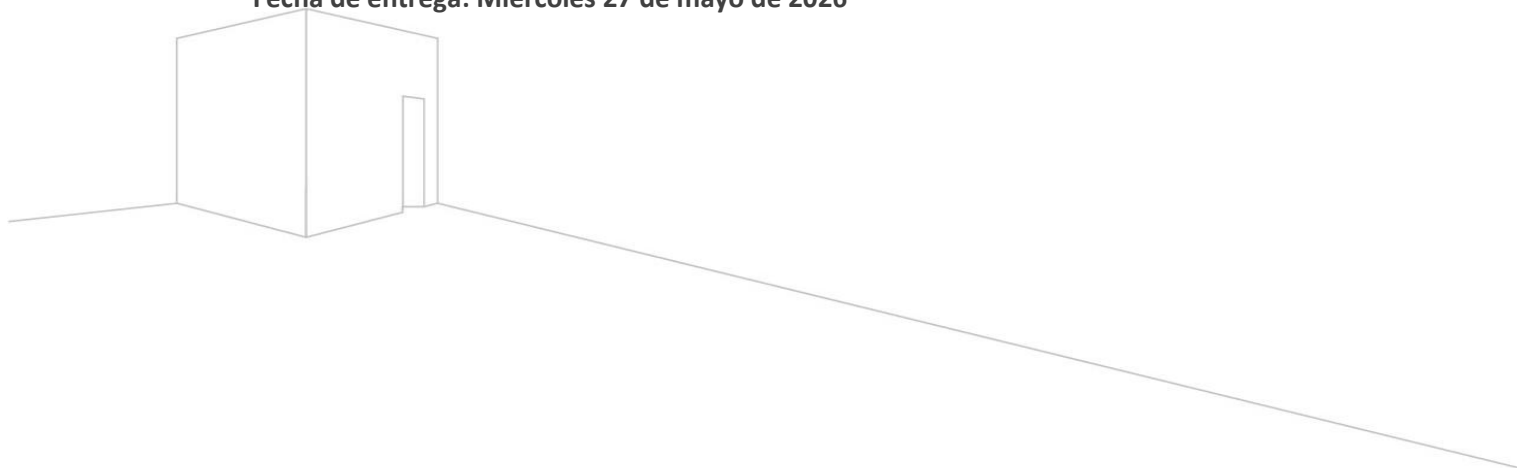
Asignatura: Proyecto de Título

Sección: TIHI84

Académico guía: José Luis Martínez Opazo

Integrantes del equipo: Cesar Contreras Molina-Sebastián Maulén-Victel Merzier

Fecha de entrega: Miércoles 27 de mayo de 2026



Contenido

Introducción	5
Detalle de las Tecnologías a implementar	6
2.1 Análisis cualitativo/cuantitativo de las tecnologías que serán implementadas.	6
2.2 Herramientas, aplicaciones, lenguajes y componentes que serán implementados.	10
Detalle de la Arquitectura a implementar.....	12
3.1.1 BPMN 1: Ingreso y registro de bio-insumos.....	13
3.1.2 BPMN 2: Monitoreo ambiental y generación de alertas.....	14
3.1.3 BPMN 3: Monitoreo ambiental y generación de alertas.....	15
3.2 Diagrama de caso de uso.....	17
3.2.1 Actores principales	17
3.2.2 Casos de uso principales	18
3.2.3 Descripción formal de casos de uso críticos.....	18
3.2.4 Descripción general de interacción	20
3.3 Diagrama de componentes	21
3.3.2 Relación entre componentes	22
3.3.3 Descripción funcional de componentes	22
3.3.4 Descripción general del diagrama	23
3.4 Modelo de datos	24
3.5 Topología de comunicaciones	28
3.5.2 Protocolos utilizados	29
3.5.3 Flujo de comunicación.....	29
3.5.4 Comunicación en tiempo real	30
3.5.5 Seguridad de las comunicaciones	30
3.6 Diagrama de Infraestructura	31
3.6.1 Componentes de infraestructura	31
3.6.2 Distribución de infraestructura	32
3.6.3 Contenedorización de servicios.....	32
3.6.4 Infraestructura cloud.....	33
3.6.7 Descripción general de infraestructura.....	34
3.7 Diagrama de Arquitectura	35
3.7.1 Capas de la arquitectura.....	35

3.7.2 Descripción de la arquitectura	36
3.7.3 Flujo general de la arquitectura	36
3.7.4 Mecanismos de seguridad arquitectónica	37
3.7.5 Descripción general del diagrama	37
Implementación de los KPI y SLA	38
4.1.1 KPI de disponibilidad de la plataforma.....	38
4.1.2 KPI de precisión del modelo de inteligencia artificial	39
4.1.3 KPI de pérdida de paquetes IoT	40
4.1.4 KPI de tiempo de respuesta de alertas.....	40
4.1.5 KPI de precisión de inventario.....	41
4.2 Descripción de SLA	41
Plan de pruebas.....	43
5.1.1 Pruebas unitarias.....	44
5.1.2 Pruebas de integración.....	44
5.1.3 Pruebas API y contratos	44
5.1.4 Pruebas End-to-End (E2E)	45
5.1.5 Pruebas de carga y rendimiento	45
5.1.7 Descripción general del plan de pruebas	46
5.2 Normas y Estándares.....	46
Gestión de Disponibilidad	47
6.1.1 Arquitectura de alta disponibilidad.....	47
6.1.2 Redundancia y persistencia de datos	47
6.1.3 Monitoreo proactivo	48
6.1.4 Recuperación automática de servicios.....	48
6.1.5 Justificación	48
6.2 Gestión de continuidad	48
6.2.1 Plan de recuperación ante desastres	49
6.2.2 Estrategia de failover.....	49
6.2.3 Continuidad de telemetría IoT	49
6.2.4 Backups y recuperación de datos.....	49
6.2.5 Recuperación mediante contenedores	50
6.2.6 Justificación	50

6.3 Plan de mantención.....	50
6.3.1 Mantenimiento preventivo	51
6.3.2 Mantenimiento correctivo	51
6.3.3 Mantenimiento evolutivo.....	51
6.3.4 Mantenimiento de seguridad.....	52
6.3.5 Optimización de rendimiento.....	52
6.3.6 Justificación	52
Revisión de Carta Gantt y fases del proyecto.....	52
7.1.1 Fase 1: Análisis e iniciación.....	53
7.1.2 Fase 2: Diseño de arquitectura.....	53
7.1.3 Fase 3: Construcción e integración	53
7.1.4 Fase 4: QA y despliegue.....	54
7.1.5 Justificación	54
Referencia a anexo	54
Conclusiones.....	55
Referencias bibliográficas.....	56

Introducción

En el contexto actual de la industria 4.0, la gestión de inventarios en sectores críticos, como los laboratorios químicos, clínicos y el área dental, ha dejado de ser una tarea administrativa secundaria para convertirse en un pilar de la continuidad operacional y la seguridad sanitaria. Los insumos utilizados en estos entornos poseen una alta sensibilidad a variables ambientales y una caducidad técnica estricta, lo que exige una trazabilidad que los sistemas de inventario tradicionales, de carácter estático y reactivo, no han logrado satisfacer de manera eficiente.

El presente informe expone la formulación y el desarrollo de una solución tecnológica avanzada: una Plataforma de Gestión Inteligente para Inventarios Críticos. Este proyecto aborda la problemática de la pérdida de capital por mermas y la incertidumbre en la integridad de los reactivos, mediante una arquitectura robusta y moderna de microservicios políglotas, integrando las capacidades transaccionales de Spring Boot con la agilidad en la captura de datos de NestJS.

A lo largo de este documento, se detallará cómo el sistema trasciende el modelo convencional FEFO (First-Expired, First-Out) para introducir el concepto de Vencimiento Dinámico. Gracias a la ingesta de telemetría en tiempo real, el software es capaz de recalcular la prioridad de uso de los insumos ante anomalías térmicas, asegurando que el material despachado mantenga sus propiedades físico-químicas intactas. Asimismo, se incorpora una capa de Analítica Predictiva (Machine Learning) encargada de proyectar la demanda y automatizar el reabastecimiento, minimizando tanto el sobre-stock como el quiebre de inventario. Un aspecto diferenciador de esta propuesta, alineado con el perfil de auditoría del autor, es el enfoque en la integridad de los datos. Se describe la implementación de registros de auditoría inmutables mediante técnicas criptográficas de hashing, garantizando que cada evento — desde la recepción mediante códigos QR hasta el consumo final— sea trazable y no repudiable ante procesos de certificación como la norma ISO 17025.

Finalmente, el informe recorre el levantamiento de requerimientos, el marco teórico que sustenta la tecnología empleada y la metodología de trabajo ágil bajo la cual se estructuró el proyecto. Con esta solución, se pretende no solo entregar una herramienta de software, sino establecer un nuevo estándar de Arquitectura TI para la gestión de activos críticos, donde la seguridad, la predictibilidad y la transparencia sean los ejes fundameta.

Detalle de las Tecnologías a implementar

2.1 Análisis cualitativo/cuantitativo de las tecnologías que serán implementadas.

Con el objetivo de seleccionar las tecnologías más adecuadas para el desarrollo de la plataforma G.A.B.I (Gestión Analítica de Bio-Insumos), se realizó un análisis comparativo considerando criterios de factibilidad técnica, económica e implementativa, además de la compatibilidad con los requerimientos funcionales y no funcionales definidos en la etapa anterior del proyecto.

La solución requiere una arquitectura moderna capaz de soportar procesamiento en tiempo real, escalabilidad, integración con sensores IoT, analítica predictiva e integridad de datos orientada a auditoría.

Por esta razón, se evaluaron diferentes tecnologías backend, frontend, motores de bases de datos y metodologías de desarrollo.

Criterios de evaluación Backend

Criterio	Descripción	Peso
Escalabilidad	Capacidad de crecimiento horizontal y modular	25%
Procesamiento en tiempo real	Manejo de eventos, WebSockets y telemetría	20%
Integración IA	Integración con servicios predictivos y analíticos	15%
Facilidad de desarrollo	Productividad y mantenibilidad del código	15%
Costos operacionales	Costos de infraestructura y mantenimiento	10%
Seguridad	Robustez, autenticación y control de acceso	15%

Evaluación cuantitativa de Frameworks Backend

Tecnología	Escalabilidad	Procesamiento en Tiempo real	Integración Real	Facilidad de desarrollo	Costos operacionales	Seguridad	Puntaje final
Nest Js	5	5	5	5	5	4	4.85
Spring Boot	5	4	4	3	3	5	4.10
Django	3	2	4	4	4	4	3.35

Método de evaluación

La evaluación cuantitativa se realizó mediante una matriz ponderada de decisión tecnológica, asignando valores entre 1 y 5 para cada criterio definido previamente. Cada puntuación fue multiplicada por el peso relativo del criterio correspondiente, obteniendo un puntaje final normalizado para cada framework evaluado.

Resultado del análisis backend

NestJS obtuvo el mayor puntaje debido a:

- Su arquitectura modular
 - Compatibilidad con microservicios
 - Soporte nativo para WebSockets
 - Integración con TypeScript
 - Facilidad para implementar servicios en tiempo real orientado a eventos.
- Adicionalmente, permite reducir la complejidad operativa al centralizar gran parte de la lógica de negocio bajo un mismo ecosistema tecnológico basado en NodeJs.

Justificación de selección backend

Se seleccionó **NestJS** como framework backend principal debido a que obtuvo el mayor puntaje en la matriz de evaluación cuantitativa, destacando especialmente en escalabilidad, procesamiento en tiempo real, compatibilidad analítica e IA, facilidad de desarrollo y costos operacionales.

Su arquitectura modular permite organizar la solución en componentes independientes, facilitando la mantenibilidad, la separación de responsabilidades y la futura escalabilidad del sistema. Además, su integración nativa con TypeScript favorece el desarrollo de código más robusto, tipado y mantenible.

NestJS también ofrece soporte adecuado para WebSockets, APIs REST y arquitecturas orientadas a microservicios, características fundamentales para G.A.B.I., ya que la plataforma requiere comunicación en tiempo real, integración con sensores IoT, generación de alertas, trazabilidad de eventos y conexión con un módulo de inteligencia artificial desarrollado en Python/FastAPI.

Si bien Spring Boot presenta mayor madurez empresarial y Django permite un desarrollo rápido, NestJS representa la alternativa más equilibrada para este proyecto, al combinar productividad, escalabilidad, integración moderna y menor complejidad operativa dentro del ecosistema tecnológico seleccionado.

Criterios de evaluación Frontend

Criterio	Descripción	Ponderación
Arquitectura SPA	Capacidad para construir aplicaciones Single Page Application modernas y escalables	25%
Reutilización de componentes	Modularidad y facilidad de reutilización de interfaces	20%
Ecosistema y comunidad	Disponibilidad de librerías, documentación y soporte empresarial	20%
Rendimiento	Fluidez, optimización de renderizado y experiencia de usuario	20%
Compatibilidad móvil y PWA	Soporte responsive y compatibilidad con Progressive Web Apps	15%

Evaluación cuantitativa de frameworks Frontend

Framework	Arquitectura SPA	Componentes reutilizables	Ecosistema y comunidad	Rendimiento	Compatibilidad PWA	Puntaje final
React	5	5	5	5	5	5.00
Angular	5	4	5	4	4	4.40
Vue.js	4	5	3	4	4	4.00

Método de evaluación

La evaluación cuantitativa de frameworks frontend se realizó mediante una matriz ponderada basada en criterios de arquitectura SPA, reutilización de componentes, ecosistema tecnológico, rendimiento y compatibilidad móvil/PWA. Cada tecnología fue evaluada mediante puntuaciones entre 1 y 5, considerando los requerimientos de visualización analítica, interacción en tiempo real y capacidad de integración con arquitecturas modernas basadas en microservicios.

Justificación de selección frontend

React fue seleccionado como framework frontend principal debido a su arquitectura basada en componentes reutilizables, alta compatibilidad con aplicaciones SPA (Single Page Application) y excelente integración con APIs REST y WebSockets. Además, presenta un ecosistema ampliamente adoptado a nivel empresarial, permitiendo construir dashboards interactivos, interfaces dinámicas y visualizaciones en tiempo real, elementos fundamentales para la plataforma G.A.B.I.

Adicionalmente, React permite compatibilidad con Progressive Web Apps (PWA), facilitando el acceso desde dispositivos móviles utilizados por operadores de laboratorio y personal de bodega.

Criterio de evaluación de Bases de Datos

Criterio	Descripción	Ponderación
Integridad transaccional	Capacidad para asegurar consistencia y cumplimiento ACID	25%
Escalabilidad	Capacidad de crecimiento y manejo de grandes volúmenes de datos	20%
Rendimiento analítico	Capacidad para consultas complejas y procesamiento analítico	20%
Consistencia de datos	Fiabilidad y precisión en operaciones concurrentes	20%

Compatibilidad y soporte empresarial	Soporte enterprise, documentación y herramientas administrativas	15%
--------------------------------------	--	-----

Evaluación cuantitativa de Base de datos

Base de datos	Integridad transaccional	Escalabilidad	Rendimiento analítico	Consistencia de datos	Compatibilidad empresarial	Puntaje final
PostgreSQL	5	5	5	5	5	5.00
MySQL	4	4	3	4	4	3.85
MongoDB	2	5	4	2	3	3.20

Método de evaluación

La evaluación cuantitativa de motores de bases de datos se realizó mediante una matriz ponderada basada en criterios de integridad transaccional, escalabilidad, rendimiento analítico, consistencia de datos y compatibilidad empresarial. Cada tecnología fue evaluada mediante puntuaciones entre 1 y 5, considerando los requerimientos de trazabilidad, auditoría, procesamiento analítico y confiabilidad operacional exigidos por la plataforma G.A.B.I.

Justificación de selección Base de datos

PostgreSQL fue seleccionado como motor de base de datos principal debido a que obtuvo el mayor puntaje en la evaluación cuantitativa, destacando especialmente en integridad transaccional, consistencia de datos y capacidades analíticas avanzadas.

La plataforma G.A.B.I. requiere almacenar información crítica relacionada con bio-insumos, lotes, movimientos de inventario, telemetría ambiental y registros de auditoría, por lo que resulta fundamental garantizar operaciones ACID, alta confiabilidad y trazabilidad completa de la información.

Además, PostgreSQL ofrece un alto nivel de compatibilidad con arquitecturas modernas basadas en microservicios, soporte para consultas complejas y una sólida capacidad de escalabilidad empresarial. Estas características lo convierten en una alternativa adecuada para entornos críticos donde la disponibilidad, consistencia y seguridad de los datos representan factores esenciales para la operación del sistema.

Conclusión general del análisis tecnológico

A partir del análisis cualitativo y cuantitativo realizado, se concluye que la combinación tecnológica más adecuada para el desarrollo de la plataforma G.A.B.I. corresponde a React, NestJS y PostgreSQL. Esta

selección responde a los requerimientos funcionales y no funcionales del proyecto, especialmente en términos de escalabilidad, procesamiento en tiempo real, trazabilidad, integridad de datos, compatibilidad con servicios de inteligencia artificial y facilidad de mantenimiento.

NestJS permite implementar una capa backend modular, segura y orientada a servicios; React facilita la construcción de una interfaz web responsiva e interactiva; y PostgreSQL garantiza consistencia transaccional, robustez analítica y confiabilidad en el almacenamiento de información crítica. En conjunto, estas tecnologías permiten construir una solución moderna, escalable y alineada con las necesidades operativas de laboratorios que gestionan bio-insumos sensibles.

2.2 Herramientas, aplicaciones, lenguajes y componentes que serán implementados.

La plataforma G.A.B.I. contempla el uso de diversas herramientas, lenguajes, servicios y componentes tecnológicos orientados al desarrollo, despliegue, integración, operación y monitoreo de la solución. Estas tecnologías fueron seleccionadas considerando su compatibilidad con la arquitectura propuesta, su capacidad de integración con servicios en tiempo real, su soporte para analítica predictiva y su aplicabilidad en entornos críticos de laboratorio.

Herramientas, lenguajes y frameworks de desarrollo

Categoría	Tecnología	Función dentro del proyecto
Frontend	React	Desarrollo de la interfaz web responsiva, dashboards y formularios operativos.
Lenguaje frontend	TypeScript	Tipado estático y mayor mantenibilidad del código.
Backend	NestJs	Implementación de la lógica de negocio, inventario, usuarios, alertas y auditoría.
Lenguaje backend	TypeScript	Desarrollo estructurado y seguro del backend.
Microservicio IA	Python / FastAPI	Predicción de demanda, análisis de riesgo y apoyo al FEFO dinámico
Base de datos	PostgreSQL	Persistencia relacional de productos, lotes, usuarios, inventario y auditoría.
Eventos y caché	Redis	Gestión de eventos, alertas en tiempo real y optimización de consultas.
Documentación API	Swagger / OpenAPI	Documentación de endpoints REST para pruebas e integración.
Control de versiones	Git / GitHub	Gestión colaborativa del código fuente.
Pruebas API	Postman	Validación manual y funcional de endpoints.

Editor de código	Visual Studio Code	Entorno principal de desarrollo (IDE)
-------------------------	--------------------	---------------------------------------

Herramienta de despliegue, integración y operación

Categoría	Tecnología / Servicio	Función dentro del proyecto
Contenedorización	Docker	Empaquetamiento y ejecución aislada de servicios.
Integración continua	GitHub Actions	Automatización de pruebas, construcción y despliegue.
Infraestructura cloud	AWS EC2	Ejecución de servicios backend, frontend y microservicio IA.
Base de datos administrada	AWS RDS	Alojamiento administrado de PostgreSQL.
Seguridad perimetral	Cloudflare	Protección DNS, CDN y mitigación básica de ataques.
Cifrado de comunicaciones	HTTPS / TLS	Protección de datos en tránsito.
Autenticación	JWT	Control de acceso seguro basado en tokens.
Almacenamiento	AWS S3 o Storage Cloud	Respaldo de archivos, reports y evidencia documental.

Componentes de hardware y dispositivo de operación

Componente	Función dentro del proyecto
Servidor cloud	Hospedaje de servicios, APIs, base de datos y microservicios
Estaciones de trabajo	Acceso administrativo y operativo al sistema
Smartphones o tablets	Escaneo de códigos QR/barra y operación móvil en laboratorio
Sensores IoT	Captura de temperatura y humedad ambiental
Red local / Internet	Comunicación entre usuarios, sensores y servicios cloud.

Servicios de TI asociados

Servicio TI	Descripción
Servicio de autenticación	Controla el inicio de sesión, roles y permisos de usuario.
Servicio de inventario	Gestiona productos, lotes, stock, ubicaciones y movimientos.
Servicio de alertas	Genera advertencias ante vencimientos, anomalías ambientales o riesgos.
Servicio de telemetría	Recibe y procesa datos de sensores IoT
Servicio de inteligencia artificial	Ejecuta predicciones de demanda y análisis de prioridad FEFO.

Servicio de auditoría	Registra acciones críticas realizadas por usuarios y sistema.
Servicio de reportabilidad	Permite consultar indicadores, dashboards y reportes operacionales.

Justificación general

La selección de estas herramientas permite construir una solución modular, escalable y mantenible, alineada con los requerimientos de trazabilidad, seguridad, monitoreo ambiental y analítica predictiva definidos para G.A.B.I.

El uso de React, NestJS, FastAPI y PostgreSQL permite separar responsabilidades entre interfaz, lógica de negocio, procesamiento analítico y persistencia de datos. A su vez, Docker, GitHub Actions y AWS facilitan el despliegue controlado de la solución, mientras que Redis, WebSockets, JWT y HTTPS fortalecen la comunicación en tiempo real, la seguridad y la operación continua del sistema.

Con esta estructura, la plataforma puede evolucionar progresivamente hacia un entorno productivo, incorporando nuevas funcionalidades, sensores adicionales, módulos predictivos y mecanismos de monitoreo sin comprometer la estabilidad general de la arquitectura.

Detalle de la Arquitectura a implementar

En este apartado se describe la arquitectura propuesta para la plataforma G.A.B.I. — Gestión Analítica de Bio-Insumos, considerando los principales procesos de negocio, componentes de software, modelo de datos, topología de comunicaciones, infraestructura tecnológica y arquitectura general de la solución.

La plataforma será implementada mediante una arquitectura web modular y distribuida, diseñada para soportar procesamiento en tiempo real, integración con sensores IoT, monitoreo ambiental, trazabilidad operativa y analítica predictiva aplicada a la gestión de bio-insumos críticos.

La solución considera React como tecnología principal para la capa de presentación, permitiendo construir interfaces web responsivas y dashboards interactivos. Para la capa de negocio se utilizará NestJS, encargado de centralizar la lógica operacional, autenticación, auditoría, gestión de inventario, alertas y reglas FEFO dinámicas.

Complementariamente, se incorporará un microservicio desarrollado en Python/FastAPI orientado al procesamiento analítico y predicción de demanda, permitiendo integrar capacidades de inteligencia artificial dentro de la solución. PostgreSQL será utilizado como motor de persistencia relacional para garantizar integridad transaccional y trazabilidad de la información, mientras que Redis permitirá gestionar eventos, caché y comunicación en tiempo real mediante mecanismos Pub/Sub y WebSockets.

A continuación, se presentan los principales diagramas BPMN, UML, modelos de datos y componentes de infraestructura que permiten representar la operación y estructura técnica de la plataforma G.A.B.I.

3.1 Diagrama BPMN

Para representar los procesos principales de negocio de la plataforma G.A.B.I., se utilizará la notación BPMN, permitiendo visualizar de forma clara la secuencia de actividades, responsables, decisiones y

resultados esperados dentro del flujo operativo. Estos procesos permiten comprender cómo la solución informática apoya la trazabilidad, control ambiental y gestión eficiente de bio-insumos críticos.

Los procesos modelados son:

1. Ingreso y registro de bio-insumos.
2. Monitoreo ambiental y generación de alertas.
3. Despacho de insumos mediante FEFO dinámico.

3.1.1 BPMN 1: Ingreso y registro de bio-insumos

Descripción del proceso

Este proceso representa el flujo mediante el cual un bio-insumo es recibido, validado, registrado y almacenado dentro del sistema G.A.B.I. Participan principalmente el operador de bodega y la plataforma informática, la cual valida la información ingresada, registra el lote, asigna ubicación y genera trazabilidad mediante auditoría.

Participantes del proceso

Participante	Rol dentro del proceso
Operador de bodega	Recibe, verifica y registra el bio-insumo.
Sistema G.A.B.I	Valida datos, registra inventario y genera auditoría.
Base de datos	Almacena producto, lote, ubicación y movimiento.

Flujo BPMN

Tipo BPMN	Actividad / Evento	Responsable
Evento inicial	Recepción de nuevo bio-insumo	Operador
Tarea	Verificar documentación del producto.	Operador
Tarea	Escanear código QR o código de barra	Operador
Tarea	Ingresar datos de producto, lote y vencimiento	Operador
Gateway	¿Los datos están completos y son válidos?	Sistema G.A.B.I
Tarea	Solicitar corrección de datos	Sistema G.A.B.I.
Tarea	Asignar ubicación de almacenamiento	Operador / Sistema
Gateway	¿Requiere condiciones especiales de temperatura?	Sistema G.A.B.I.
Tarea	Asociar a ubicación monitoreada	Sistema G.A.B.I
Tarea	Registrar inventario en base de datos	Sistema G.A.B.I
Tarea	Generar log de auditoría	Sistema G.A.B.I
Evento final	Bio-insumo registrado correctamente	Sistema G.A.B.I

Reglas de negocio

- Todo bio-insumo debe estar asociado a un lote y fecha de vencimiento.
- Si los datos ingresados son incompletos, el sistema debe bloquear el registro hasta su corrección.

- Si el producto requiere condiciones especiales de almacenamiento, debe asignarse a una ubicación monitoreada.
- Todo ingreso debe generar un registro de auditoría para asegurar trazabilidad.

Resultado esperado

El bio-insumo queda registrado en el sistema con su lote, ubicación, fecha de vencimiento, condiciones de almacenamiento y trazabilidad inicial.

3.1.2 BPMN 2: Monitoreo ambiental y generación de alertas

Descripción del proceso

Este proceso representa la captura, validación y monitoreo de variables ambientales críticas, principalmente temperatura y humedad, mediante sensores IoT o mecanismos de simulación integrados en la plataforma G.A.B.I.

El objetivo del proceso es detectar condiciones fuera de rango que puedan comprometer la conservación de bio-insumos sensibles, permitiendo generar alertas automáticas, recalcular prioridades FEFO y mantener trazabilidad de los eventos ambientales ocurridos dentro del laboratorio.

Participantes del proceso

Participante	Rol dentro del proceso
Sensores IoT / Simulador	Capturan información ambiental
Sistema G.A.B.I	Procesa, valida y analiza telemetría
Jefe de laboratorio	Recibe alertas y supervisa incidentes
Base de datos	Almacena registros de telemetría y alertas

Flujo BPMN

Tipo BPMN	Actividad / Evento	Responsable
Evento de inicio	Recepción de datos ambientales	Sensor IoT
Tarea	Capturar temperatura y humedad	Sensor IoT
Tarea	Enviar datos al backend	Sensor IoT / Sistema
Tarea	Validar rangos permitidos	Sistema G.A.B.I.

Gateway	¿Los valores se encuentran fuera de rango?	Sistema G.A.B.I.
Tarea	Registrar telemetría normal	Sistema G.A.B.I.
Tarea	Generar alerta crítica	Sistema G.A.B.I.
Tarea	Registrar incidente ambiental	Sistema G.A.B.I.
Tarea	Recalcular prioridad FEFO dinámica	Sistema G.A.B.I.
Tarea	Notificar al jefe de laboratorio	Sistema G.A.B.I.
Tarea	Registrar auditoría del evento	Sistema G.A.B.I.
Evento final	Estado ambiental actualizado	Sistema G.A.B.I.

Reglas de negocio

- Toda lectura ambiental debe quedar registrada en la base de datos.
- Si la temperatura o humedad supera el umbral definido, el sistema debe generar automáticamente una alerta.
- Las anomalías ambientales pueden modificar la prioridad de utilización de los lotes asociados.
- Todo evento crítico debe quedar auditado para fines de trazabilidad y control de calidad.
- Las alertas deben notificarse en tiempo real mediante WebSockets y dashboard operacional.

Resultado esperado

El sistema monitorea continuamente las condiciones ambientales del laboratorio, detectando anomalías de manera temprana y permitiendo tomar decisiones preventivas sobre bio-insumos críticos, reduciendo riesgos de degradación y pérdidas operacionales.

3.1.3 BPMN 3: Monitoreo ambiental y generación de alertas

Descripción del proceso

Este proceso representa la salida controlada de bio-insumos desde bodega o laboratorio, utilizando un mecanismo FEFO dinámico capaz de priorizar automáticamente los lotes más críticos considerando fecha de vencimiento, condiciones ambientales históricas y riesgo de degradación.

El objetivo es garantizar que los bio-insumos sean utilizados en el orden más seguro y eficiente posible, minimizando pérdidas operacionales, vencimientos y riesgos asociados al deterioro de materiales sensibles.

Participantes del proceso

Participante	Rol dentro del proceso
Operador de bodega	Solicita y ejecuta el despacho del bio-insumo.
Sistema G.A.B.I	Evalúa disponibilidad y aplica lógica FEFO dinámica
Módulo IA	Analiza riesgo y prioridad de salida
Base de datos	Registra movimientos y trazabilidad

Flujo BPMN

Tipo BPMN	Actividad / Evento	Responsable
Evento de inicio	Solicitud de despacho de bio-insumo	Operador
Tarea	Buscar producto solicitado	Sistema G.A.B.I
Tarea	Consultar lotes disponibles	Sistema G.A.B.I.
Tarea	Evaluar fecha de vencimiento	Sistema G.A.B.I
Tarea	Analizar condiciones ambientales históricas	Sistema G.A.B.I. / IA
Tarea	Aplicar regla FEFO dinámica	Sistema G.A.B.I
Gateway	¿El lote seleccionado corresponde al recomendado?	Sistema G.A.B.I.
Tarea	Generar advertencia o bloqueo	Sistema G.A.B.I.
Tarea	Autorizar despacho	Sistema G.A.B.I
Tarea	Actualizar stock e inventario	Sistema G.A.B.I
Tarea	Registrar movimiento de inventario	Sistema G.A.B.I
Tarea	Generar auditoría del despacho	Sistema G.A.B.I
Evento final	Bio-insumo despachado correctamente	Sistema G.A.B.I

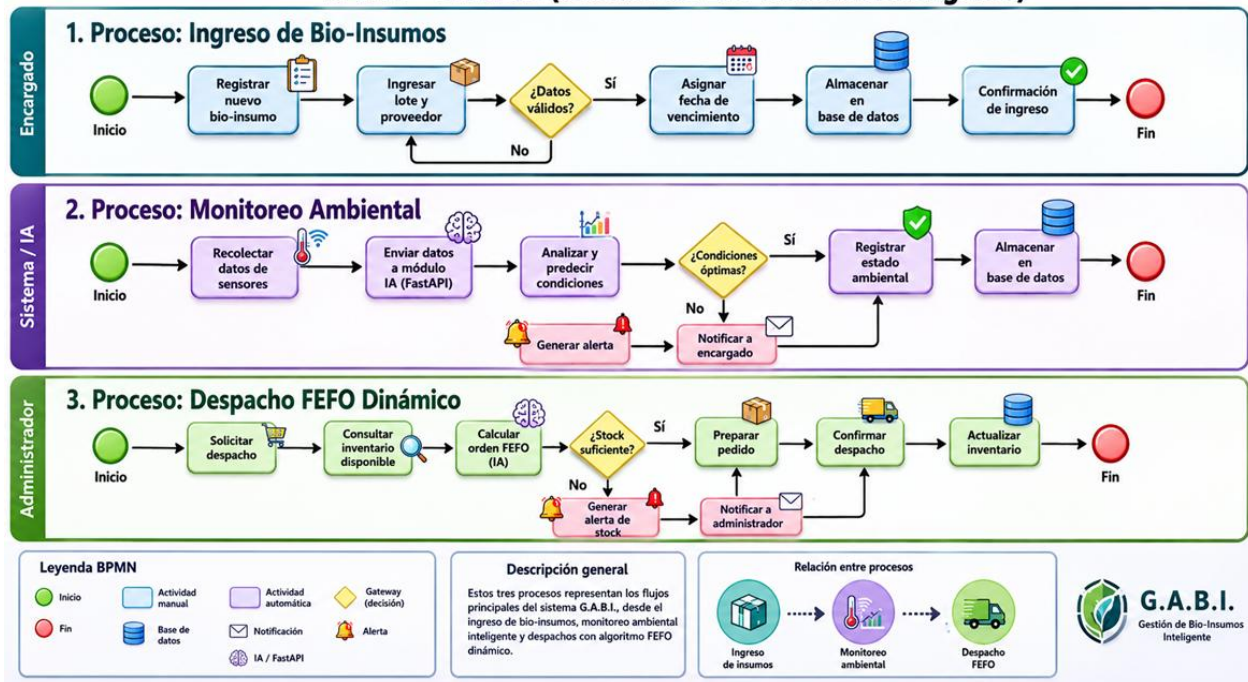
Reglas de negocio

- El sistema debe priorizar automáticamente el lote con menor vida útil disponible.
- Las condiciones ambientales históricas pueden alterar la prioridad FEFO de un lote.
- Si el operador intenta seleccionar un lote distinto al recomendado, el sistema podrá generar advertencias o bloquear la operación dependiendo de la criticidad.
- Todo despacho debe actualizar automáticamente el inventario y generar trazabilidad auditada.
- Los movimientos realizados deben quedar registrados para futuras revisiones de calidad y cumplimiento normativo.

Resultado esperado

El sistema permite ejecutar despachos inteligentes y controlados de bio-insumos, optimizando el uso de lotes críticos y reduciendo riesgos de vencimiento, deterioro y pérdidas operacionales mediante un mecanismo FEFO dinámico apoyado por monitoreo ambiental e inteligencia artificial.

BPMN – G.A.B.I. (Gestión de Bio-Insumos Inteligente)



3.2 Diagrama de caso de uso

Los diagramas de casos de uso permiten representar la interacción entre los distintos actores del sistema y las funcionalidades principales de la plataforma G.A.B.I. Su objetivo es modelar el comportamiento funcional de la solución desde el punto de vista de los usuarios y componentes externos que interactúan con el sistema.

Mediante esta representación es posible identificar los procesos operativos, administrativos y analíticos involucrados en la gestión de bio-insumos, así como las relaciones entre usuarios, módulos del sistema y servicios de inteligencia artificial.

3.2.1 Actores principales

Actor	Descripción
Operador de bodega	Usuario encargado de registrar ingresos, despachos y movimientos de bio-insumos
Jefe de laboratorio	Usuario responsable de supervisar alertas, indicadores y procesos críticos
Auditor / Calidad	Usuario encargado de revisar trazabilidad, auditoría e integridad de registros
Administrador del sistema	Usuario encargado de gestionar usuarios, roles y configuraciones
Sistema IA	Servicio encargado de generar predicciones y análisis FEFO dinámicos

Sensores IoT	Dispositivos encargados de capturar datos ambientales
--------------	---

3.2.2 Casos de uso principales

Caso de uso	Actor principal	Descripción
Inicio sesión	Todos los usuarios	Permite acceder al sistema mediante autenticación JWT
Registrar bio-insumo	Operador de bodega	Permite registrar nuevos productos y lotes
Escanear código QR/Barra	Operador de bodega	Permite identificar bio-insumos mediante dispositivos móviles
Consultar inventario	Operador / Jefe	Permite visualizar stock y disponibilidad
Registrar despacho FEFO	Operador de bodega	Ejecuta salida de productos bajo criterio FEFO dinámico
Monitorear telemetría	Jefe / Operador	Permite visualizar temperatura y humedad
Generar alertas	Sistema G.A.B.I	Detecta anomalías ambientales y operacionales
Consultar dashboard	Jefe de laboratorio	Permite visualizar indicadores y métricas
Revisar auditoría	Auditor / Calidad	Permite revisar logs y trazabilidad
Generar predicción de stock	Sistema IA	Ejecuta análisis predictivo de demanda
Gestionar usuarios	Administrador	Administra permisos y accesos

3.2.3 Descripción formal de casos de uso críticos

Caso de uso 1: Registrar bio-insumo

Elemento	Descripción
Caso de uso	Registrar bio-insumo
Actor principal	Operador de bodega
Objetivo	Registrar un nuevo bio-insumo dentro del sistema
Precondición	Usuario autenticado con permisos operativos
Postcondición	Producto, lote y ubicación registrados en inventario
Flujo principal	Escaneo → Validación → Registro → Auditoría
Flujo alternativo	Datos incompletos o lote duplicado.

Descripción del flujo

El operador de bodega accede al módulo de inventario y registra un nuevo bio-insumo mediante escaneo QR o ingreso manual. El sistema valida la información ingresada, verifica la existencia del lote y registra el

producto junto con su fecha de vencimiento, ubicación física y condiciones especiales de almacenamiento. Finalmente, el sistema genera un registro de auditoría asociado a la operación realizada.

Caso de uso 2: Registrar despacho mediante FEFO dinámico

Elemento	Descripción
Caso de uso	Registrar despacho FEFO
Actor principal	Operador de bodega
Objetivo	Ejecutar salida controlada de bio-insumos
Precondición	Existencia de stock disponible
Postcondición	Inventario actualizado y movimiento registrado
Flujo principal	Solicitud → Evaluación FEFO → Despacho → Auditoría
Flujo alternativo	Selección de lote no recomendado

Descripción del flujo

El operador solicita el despacho de un bio-insumo desde el sistema. G.A.B.I. consulta los lotes disponibles y aplica la lógica FEFO dinámica considerando fecha de vencimiento, historial ambiental y nivel de riesgo asociado. Posteriormente, el sistema recomienda el lote óptimo para despacho. Si el operador intenta seleccionar otro lote distinto al recomendado, el sistema puede generar advertencias o bloquear la operación según la criticidad configurada.

Una vez autorizado el despacho, el inventario se actualiza automáticamente y se registra la trazabilidad completa del movimiento realizado.

Caso de uso 3: Monitorear telemetría ambiental

Elemento	Descripción
Caso de uso	Monitorear telemetría
Actor principal	Jefe de laboratorio
Objetivo	Supervisar condiciones ambientales de almacenamiento
Precondición	Sensores IoT conectados y operativos
Postcondición	Datos ambientales almacenados y analizados
Flujo principal	Captura → Validación → Visualización
Flujo alternativo	Detección de anomalías ambientales

Descripción del flujo

Los sensores IoT capturan periódicamente datos de temperatura y humedad desde las zonas de almacenamiento. El sistema recibe la telemetría, valida los rangos permitidos y almacena la información en la base de datos. Los usuarios autorizados pueden visualizar esta información mediante dashboards y gráficos en tiempo real.

En caso de detectar valores fuera de rango, el sistema genera alertas automáticas y activa mecanismos de auditoría y seguimiento.

Caso de uso 4: Generar alertas automáticas

Elemento	Descripción
----------	-------------

Caso de uso	Generar alertas
Actor principal	Sistema G.A.B.I
Objetivo	Detectar eventos críticos y notificar incidentes
Precondición	Existencia de reglas configuradas
Postcondición	Alerta registrada y notificada
Flujo principal	Monitoreo → Detección → Notificación
Flujo alternativo	Reintento de notificación

Descripción del flujo

El sistema monitorea continuamente eventos operacionales y ambientales asociados a bio-insumos, sensores y procesos críticos. Cuando una condición supera los umbrales establecidos, G.A.B.I. genera automáticamente una alerta crítica y la envía mediante WebSockets y dashboards operacionales.

Todas las alertas quedan registradas para fines de trazabilidad, auditoría y control de calidad.

Caso de uso 5: Consultar dashboard operacional

Elemento	Descripción
Caso de uso	Consultar dashboard
Actor principal	Jefe de laboratorio
Objetivo	Visualizar indicadores operacionales y analíticos
Precondición	Usuario autenticado
Postcondición	Información visualizada correctamente
Flujo principal	Consulta → Procesamiento → Visualización
Flujo alternativo	Error de carga o pérdida de conexión

Descripción del flujo

El jefe de laboratorio accede al dashboard principal de G.A.B.I. para revisar indicadores críticos relacionados con inventario, vencimientos, alertas ambientales, estado de sensores y predicciones de demanda. El sistema procesa la información almacenada y genera visualizaciones dinámicas en tiempo real, facilitando la toma de decisiones operacionales y preventivas.

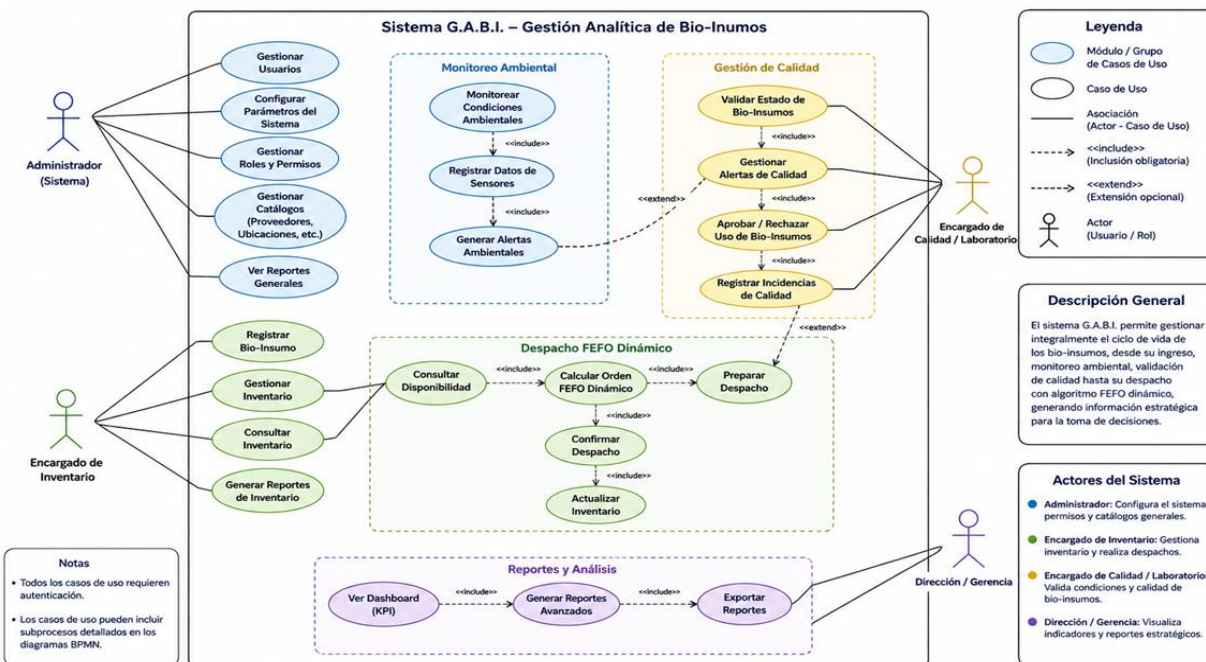
3.2.4 Descripción general de interacción

Los casos de uso modelados permiten representar el comportamiento funcional de la plataforma G.A.B.I. desde una perspectiva operacional y administrativa. La solución integra usuarios humanos, sensores IoT y servicios de inteligencia artificial dentro de un entorno centralizado orientado a la trazabilidad, monitoreo ambiental y gestión inteligente de bio-insumos.

La interacción entre los distintos actores y componentes permite automatizar procesos críticos de laboratorio, reducir riesgos asociados al vencimiento o degradación de materiales sensibles y mejorar la capacidad de supervisión y control mediante mecanismos de auditoría, alertas y analítica predictiva.

DIAGRAMA UML – CASOS DE USO

Sistema G.A.B.I. – Gestión Analítica de Bio-Insumos



3.3 Diagrama de componentes

El diagrama de componentes permite representar los módulos principales que conforman la plataforma G.A.B.I., así como las relaciones de dependencia e interacción entre los distintos servicios, capas lógicas y componentes tecnológicos que participan en la operación del sistema.

La arquitectura propuesta adopta un enfoque modular y desacoplado, permitiendo separar responsabilidades entre presentación, lógica de negocio, procesamiento analítico, persistencia y comunicación en tiempo real.

3.3.1 Componentes lógicos del sistema

Componente	Tecnología	Responsabilidad
Frontend Web	React + TypeScript	Interfaz de usuario, dashboards, formularios y visualización operacional
API Backend	NestJS	Gestión de inventario, usuarios, autenticación, auditoría y reglas FEFO
Microservicio IA	Python + FastAPI	Predicción de demanda y análisis inteligente de bio-insumos
Base de Datos	PostgreSQL	Persistencia relacional de productos, lotes, movimientos y auditoría
Motor de eventos	Redis	Caché, mensajería y distribución de eventos en tiempo real

Gateway WebSocket	NestJS Gateway	Comunicación bidireccional de alertas y telemetría
Servicio de autenticación	JWT	Control de acceso y autorización segura
Documentación API	Swagger/OpenAPI	Documentación y pruebas de endpoints REST
Sensores IoT	Dispositivos externos	Captura de temperatura y humedad
CI/CD	Github Actions	Automatización de integración y despliegue

3.3.2 Relación entre componentes

Componente origen	Componente destino	Tipo de interacción
Frontend React	Backend NestJS	API REST HTTPS
Frontend React	Gateway WebSocket	Comunicación en tiempo real
Backend NestJS	PostgreSQL	Persistencia de datos
Backend NestJS	Redis	Eventos, caché y Pub/Sub
Backend NestJS	FastAPI	Solicitud de análisis predictivo
Sensores IoT	Backend NestJS	Envío de telemetría
Backend NestJS	JWT	Validación de autenticación
GitHub Actions	AWS/Docker	Despliegue automatizado

3.3.3 Descripción funcional de componentes

Frontend React

Corresponde a la capa de presentación del sistema, encargada de permitir la interacción entre usuarios y la plataforma G.A.B.I. Incluye dashboards, formularios, módulos de inventario, alertas y visualización de indicadores operacionales en tiempo real.

Backend NestJS

Representa el núcleo operacional de la solución. Centraliza la lógica de negocio asociada a:

- Inventario
- Gestión de lotes
- FEFO dinámico
- Autenticación
- Auditoria
- Alertas
- Usuarios
- Trazabilidad del sistema

Además, expone endpoints REST y administra eventos mediante WebSockets y Redis.

Microservicio IA

Servicio independiente desarrollado en Python/FastAPI encargado de ejecutar procesos analíticos y predictivos relacionados con:

- Proyección de demanda
- Riesgo de vencimiento
- Análisis de degradación

- Apoyo a la lógica FEFO dinámica

PostgreSQL

Motor de persistencia relacional utilizado para almacenar información crítica del sistema, incluyendo:

- Productos
- Lotes
- Inventario
- Movimiento
- Telemetría
- Usuarios
- Auditoría
- Alertas

Redis

Componentes utilizados para:

- Caché
- Mensajería
- Distribución de eventos
- Soporte de comunicación en tiempo real mediante mecanismos Pub/Sub

Gateway WebSocket

Permite distribuir eventos críticos y alertas operacionales en tiempo real hacia los clientes conectados, mejorando la capacidad de monitoreo y supervisión del laboratorio.

Diagramas de caso de uso.

3.3.4 Descripción general del diagrama

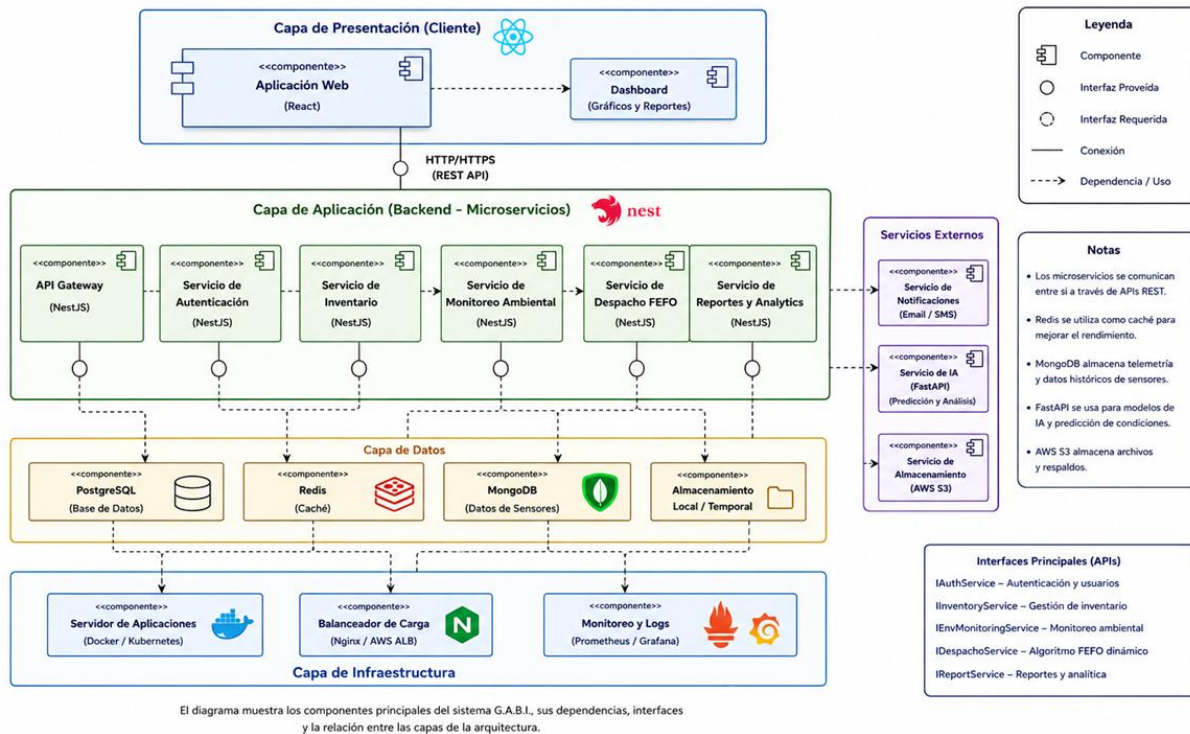
La plataforma G.A.B.I. se estructura mediante una arquitectura modular basada en componentes desacoplados que interactúan mediante APIs REST, eventos Pub/Sub y WebSockets.

El frontend desarrollado en React consume los servicios expuestos por NestJS, mientras que el backend administra la lógica transaccional y coordina la interacción con PostgreSQL, Redis y el microservicio analítico desarrollado en FastAPI.

Esta separación permite mejorar la mantenibilidad, escalabilidad y disponibilidad del sistema, facilitando futuras integraciones con sensores IoT, módulos de inteligencia artificial y servicios cloud distribuidos.

DIAGRAMA UML DE COMPONENTES

Sistema G.A.B.I. – Gestión Analítica de Bio-Insumos



3.4 Modelo de datos

El modelo de datos de G.A.B.I. se basa en una estructura relacional implementada en PostgreSQL, debido a que la plataforma requiere integridad transaccional, trazabilidad, consistencia de datos y relaciones claras entre productos, lotes, inventario, usuarios, sensores, telemetría, alertas y auditoría.

El diseño considera entidades orientadas a soportar los procesos principales del sistema: registro de bio-insumos, control de lotes, monitoreo ambiental, despacho bajo criterio FEFO dinámico, generación de alertas, predicción de demanda y control de acciones mediante auditoría.

Entidades principales

Entidad	Descripción
Usuario	Registra los usuarios que acceden al sistema
Rol	Define perfiles de acceso y permisos
Producto	Representa los bio-insumos gestionados
Lote	Registra lotes, fechas de vencimiento y estado
Ubicación	Define zonas físicas de almacenamiento

Inventario	Controla stock por producto, lote y ubicación
MovimientoInventario	Registra ingresos, despachos, ajustes y transferencias
Sensor	Representa sensores físicos o simulados
Telemetría	Registra temperatura, humedad y lecturas ambientales
Alerta	Registra eventos críticos o advertencias
Auditoría	Registra acciones relevantes realizadas en el sistema
PredicciónStock	Almacena resultados del módulo de inteligencia artificial

Relaciones principales del modelo

Relación	Descripción
Usuario - Rol	Un usuario pertenece a un rol y un rol puede estar asociado a varios usuarios
Producto - Lote	Un producto puede tener múltiples lotes asociados
Lote - Inventario	Un lote puede estar presente en una o más ubicaciones de inventario
Ubicación – Inventario	Una ubicación puede contener distintos productos y lotes
Inventario – MovimientoInventario	Cada movimiento afecta un registro de inventario
Sensor – Telemetría	Un sensor puede generar múltiples lecturas ambientales
Telemetría – Alerta	Una lectura fuera de rango puede generar una alerta

Producto – PredicciónStock	Un producto puede tener múltiples predicciones de stock
Usuario – Auditoría	Las acciones realizadas por usuarios quedan registradas para trazabilidad
Alerta – Lote	Una alerta puede asociarse a un lote afectado por condiciones ambientales

Descripción del modelo lógico

El modelo lógico se estructura en torno a la entidad Producto, que representa los bio-insumos registrados en la plataforma. Cada producto puede estar asociado a uno o más Lotes, los cuales contienen información relevante como código de lote, fecha de vencimiento, estado y condiciones requeridas de almacenamiento.

La entidad Inventario permite controlar la cantidad disponible de cada producto según lote y ubicación física. Este diseño permite conocer con precisión qué cantidad de un bio-insumo existe, dónde se encuentra almacenado y bajo qué lote debe ser despachado según la lógica FEFO dinámica.

Los movimientos de entrada, salida, ajuste o transferencia son almacenados en MovimientoInventario, permitiendo mantener trazabilidad completa sobre cada operación realizada en el sistema.

Para el monitoreo ambiental, la entidad Sensor registra los dispositivos físicos o simulados utilizados para capturar variables como temperatura y humedad. Estas lecturas son almacenadas en Telemetría, y cuando se detectan valores fuera de rango, se genera un registro en Alerta.

La entidad PredicciónStock almacena los resultados generados por el microservicio de inteligencia artificial, permitiendo proyectar demanda, riesgo de quiebre de stock o priorización de lotes críticos.

Finalmente, las entidades Usuario, Rol y Auditoría permiten controlar el acceso al sistema, aplicar permisos según perfil y registrar acciones relevantes para asegurar trazabilidad, seguridad e integridad de la información.

Diccionario de datos resumidos

Tabla	Campo	Tipo de dato	Descripción
usuario	Id_usuario	UUID	Identificador único del usuario
usuario	nombre	VARCHAR(100)	Nombre del usuario
usuario	correo	VARCHAR(150)	Correo utilizado para autenticación
usuario	password_hash	VARCHAR(255)	Contraseña cifrada
usuario	id_rol	UUID	Rol asociado al usuario
rol	id_rol	UUID	Identificador único del rol

rol	nombre	VARCHAR(50)	Nombre del rol
producto	id_producto	UUID	Identificador único del producto
producto	nombre	VARCHAR(150)	Nombre del bio-insumo
producto	categoría	VARCHAR(100)	Categoría del bio-insumo
lote	id_lote	UUID	Identificador único del lote
lote	id_producto	UUID	Producto asociado al lote
lote	codigo_lote	VARCHAR(100)	Código del lote
lote	fecha_vencimiento	DATE	Fecha de vencimiento del lote
lote	estado	VARCHAR(50)	Estado del lote
ubicacion	id_ubicacion	UUID	Identificador de la ubicación
ubicacion	nombre	VARCHAR(100)	Nombre o código de ubicación
ubicacion	descripción	TEXT	Descripción física de la ubicación
inventario	id_inventario	UUID	Identificador del registro de inventario
inventario	id_producto	UUID	Producto asociado
inventario	id_lote	UUID	Lote asociado
inventario	id_ubicacion	UUID	Ubicación física
inventario	cantidad_disponible	INTEGER	Stock disponible
movimiento_inventario	id_movimiento	UUID	Identificador del movimiento
movimiento_inventario	id_inventario	UUID	Inventario afectado
movimiento_inventario	tipo_movimiento	VARCHAR(50)	Ingreso, salida, ajuste o transferencia
movimiento_inventario	cantidad	INTEGER	Cantidad movida
movimiento_inventario	fecha_movimiento	TIMESTAMP	Fecha y hora del movimiento
sensor	id_sensor	UUID	Identificador del sensor
sensor	codigo_sensor	VARCHAR(100)	Código del sensor
sensor	tipo	VARCHAR(50)	Tipo de sensor
telemetria	id_telemetria	UUID	Identificador de la lectura
telemetria	id_sensor	UUID	Sensor asociado
telemetria	temperatura	DECIMAL(5,2)	Temperatura registrada
telemetria	humedad	DECIMAL(5,2)	Humedad registrada
telemetria	fecha_registro	TIMESTAMP	Fecha y hora de lectura

alerta	id_alerta	UUID	Identificador de la alerta
alerta	id_lote	UUID	Lote afectado
alerta	tipo_alerta	UUID	Tipo de alerta generada
alerta	nivel_criticidad	VARCHAR(50)	Baja, media o alta
auditoria	id_auditoria	UUID	Identificador del registro
auditoria	id_usuario	UUID	Usuario que ejecutó la acción
auditoria	accion	VARCHAR(150)	Acción realizada
auditoria	fecha_accion	TIMESTAMP	Fecha y hora de la acción
prediccion_stock	id_prediccion	UUID	Identificador de la predicción
prediccion_stock	id_producto	UUID	Producto analizado
prediccion_stock	demanda_estimada	INTEGER	Demanda proyectada
prediccion_stock	riesgo_quiebre	DECIMAL(5,2)	Probabilidad de quiebre de stock
prediccion_stock	fecha_prediccion	TIMESTAMP	Fecha de generación

3.5 Topología de comunicaciones

La topología de comunicaciones de la plataforma G.A.B.I. define la forma en que interactúan los distintos componentes tecnológicos del sistema, permitiendo el intercambio seguro y eficiente de información entre usuarios, servicios backend, sensores IoT, módulos analíticos y componentes de infraestructura cloud.

La solución considera una arquitectura distribuida orientada a servicios, donde la comunicación entre componentes se realiza mediante protocolos seguros y mecanismos de transmisión en tiempo real, permitiendo soportar monitoreo ambiental, procesamiento transaccional, generación de alertas y trazabilidad operacional.

3.5.1 Componentes de comunicación

Componente	Función dentro de la comunicación
Cliente Web/Movil	Permite a los usuarios acceder a la plataforma mediante navegador o dispositivos móviles
Frontend React	Gestiona la interacción visual y consumo de APIs
Backend NestJS	Centraliza la lógica de negocio y coordinación de servicios
PostgreSQL	Almacena información persistente y transaccional

Redis	Gestiona eventos internos y comunicación Pub/Sub
Gateway WebSocket	Distribuye eventos y alertas en tiempo real
Microservicio IA	Procesa predicciones y análisis inteligentes
Sensores IoT	Capturan información ambiental
Infraestructura Cloud AWS	Hospeda servicios y recursos tecnológicos

3.5.2 Protocolos utilizados

Protocolo	Uso dentro del sistema
HTTPS	Comunicación segura entre cliente y backend
REST/JSON	Intercambio de información entre frontend y backend
WebSocket (WSS)	Comunicación bidireccional en tiempo real
TCP/IP	Comunicación base entre servicios y dispositivos
JWT	Autenticación y autorización segura
Redis Pub/Sub	Distribución de eventos internos
TLS 1.2/1.3	Cifrado de datos en tránsito
HTTP	Comunicación controlada con sensores o simuladores

3.5.3 Flujo de comunicación

Origen	Destino	Tipo de comunicación
Usuario	Frontend React	Navegación web HTTPS
Frontend React	Backend NestJS	API REST/JSON
Frontend React	Gateway WebSocket	Comunicación en tiempo real
Backend NestJS	PostgreSQL	Consultas y persistencia transaccional
Backend NestJS	Redis	Publicación y consumo de eventos
Backend NestJS	FastAPI	Solicitud de análisis predictivo
Sensores IoT	Backend NestJS	Envío de telemetría
Backend NestJS	JWT	Validación de autenticación
GitHub Actions	AWS/Docker	Automatización de despliegues

3.5.4 Comunicación en tiempo real

La plataforma incorpora mecanismos de comunicación en tiempo real mediante WebSockets y Redis Pub/Sub, permitiendo distribuir eventos críticos hacia los usuarios conectados sin necesidad de recargar manualmente la interfaz.

Este mecanismo será utilizado principalmente para:

- generación de alertas ambientales,
- vencimientos críticos,
- actualización automática de dashboards,
- monitoreo de telemetría,
- y notificaciones operacionales.

El backend NestJS actuará como coordinador principal de eventos, mientras que Redis permitirá desacoplar y distribuir mensajes internos entre componentes del sistema.

3.5.5 Seguridad de las comunicaciones

La seguridad de las comunicaciones representa un aspecto crítico dentro de la arquitectura de G.A.B.I., debido a la sensibilidad operacional y trazabilidad requerida en la gestión de bio-insumos.

Por esta razón, la solución implementará:

Mecanismo	Función
HTTPS + TLS	Cifrado de comunicaciones
JWT	Autenticación basada en tokens
Control de roes	Restricción de acceso según perfil
Validación de APIs	Protección de endpoints
WebSockets seguros (WSS)	Protección de eventos en tiempo real
Cloudflare	Protección perimetral y mitigación básica

Estas medidas permiten proteger la confidencialidad, integridad y disponibilidad de la información transmitida entre usuarios, dispositivos y servicios.

3.5.6 Descripción general de la topología

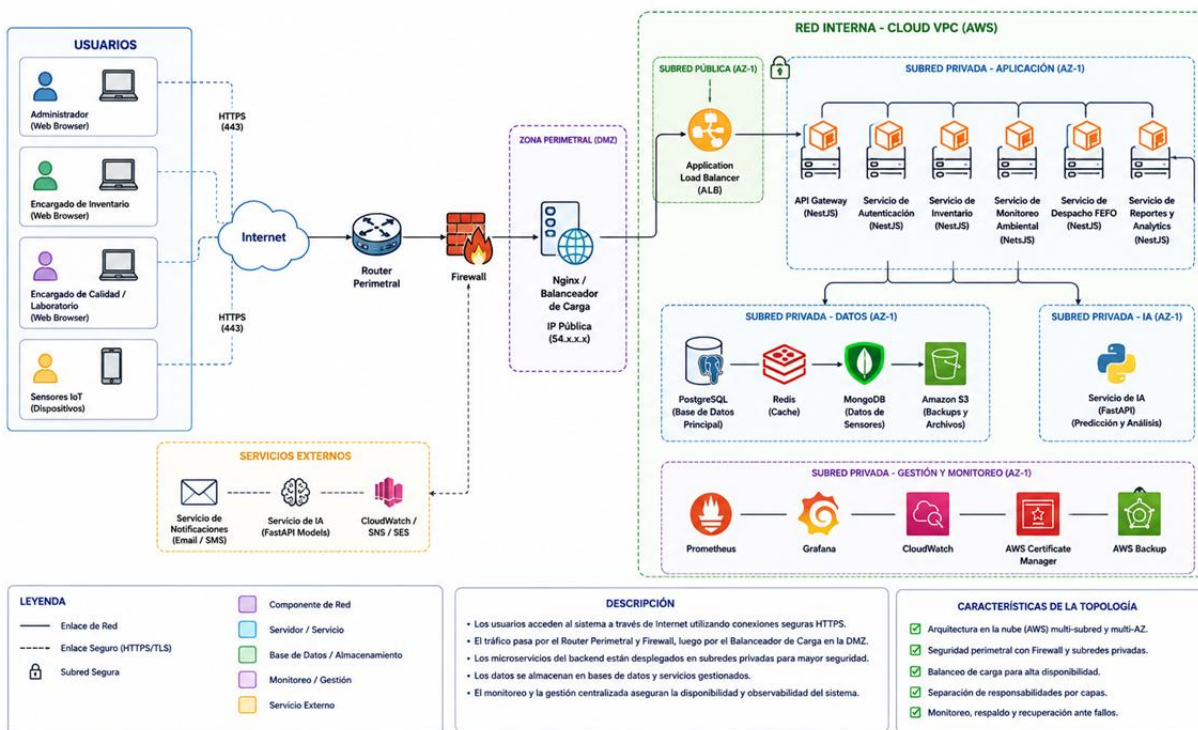
La topología propuesta corresponde a una arquitectura centralizada basada en servicios distribuidos, donde los usuarios acceden a la plataforma mediante clientes web conectados de forma segura al backend principal desarrollado en NestJS.

El backend centraliza la lógica operacional y coordina la interacción con PostgreSQL, Redis y el microservicio analítico implementado en FastAPI. Paralelamente, los sensores IoT o simuladores ambientales transmiten información de telemetría hacia la plataforma, permitiendo monitoreo continuo y generación automática de alertas.

La incorporación de WebSockets, Redis Pub/Sub y servicios cloud permite soportar comunicación en tiempo real, escalabilidad operativa y disponibilidad continua, alineándose con los requerimientos de trazabilidad, monitoreo y control exigidos en entornos críticos de laboratorio.

DIAGRAMA DE TOPOLOGÍA DE COMUNICACIONES

Sistema G.A.B.I. – Gestión Analítica de Bio-Insumos



3.6 Diagrama de Infraestructura

La infraestructura tecnológica propuesta para la plataforma G.A.B.I. fue diseñada considerando criterios de escalabilidad, disponibilidad, modularidad, seguridad y capacidad de integración con servicios distribuidos y monitoreo en tiempo real.

La solución contempla un entorno cloud basado en servicios AWS, utilizando contenedores Docker para desacoplar componentes y facilitar el despliegue, mantención y escalabilidad de la plataforma.

3.6.1 Componentes de infraestructura

Componente	Descripción
Cliente Web/Móvil	Navegadores y dispositivos utilizados por operadores y usuarios
Internet	Medio de comunicación entre usuarios y plataforma cloud
Cloudflare	Gestión DNS, CDN y protección perimetral
AWS EC2	Servidor principal de ejecución de servicios
Docker	

	Contenedorización de frontend, backend y microservicios
React Frontend	Interfaz web desplegada en contenedor
NestJS Backend	API principal y lógica de negocio
FastAPI IA	Microservicio analítico y predictivo
PostgreSQL / AWS RDS	Persistencia relacional administrada
Redis	Caché y mensajería de eventos
GitHub Actions	Integración y despliegue continuo
Storage Cloud	Respaldo de archivos y reportes
Sensores IoT	Captura de temperatura y humedad

3.6.2 Distribución de infraestructura

Capa	Componentes asociados
Capa cliente	Navegador web, dispositivos móviles
Capa de presentación	React + WebSockets
Capa de negocio	NestJS + JWT
Capa analítica	Python/FastAPI
Capa de persistencia	PostgreSQL + Redis
Capa de infraestructura	AWS EC2 + Docker
Capa de integración y CI/CD	GitHub Actions
Capa de seguridad	Cloudflare + HTTPS/TLS

3.6.3 Contenedorización de servicios

La solución utilizará Docker como mecanismo de contenedorización, permitiendo ejecutar los distintos componentes de forma desacoplada y portable.

Cada servicio podrá desplegarse en contenedores independientes:

Contenedor	Función
Frontend Container	Interfaz React
Backend Container	API NestJS
IA Container	FastAPI y procesamiento predictivo
Redis Container	Caché y eventos
PostgreSQL Container (entorno local)	Persistencia de datos

La utilización de Docker facilita:

- Despliegue controlado
- Aislamiento de dependencias
- Portabilidad
- Mantenibilidad
- Escalabilidad horizontal

3.6.4 Infraestructura cloud

La plataforma será desplegada principalmente sobre servicios AWS debido a:

- Alta disponibilidad
- Escalabilidad
- Flexibilidad operacional
- Compatibilidad con arquitecturas modernas basadas en microservicios.

Servicios cloud considerados

Servicio AWS	Función
AWS EC2	Ejecución de contenedores y servicios
AWS RDS	Base de datos PostgreSQL administrada
AWS S3	Almacenamiento de archivos y respaldos
AWS CloudWatch	Monitoreo y logs
AWS IAM	Gestión de acceso y permisos

3.6.5 Integración continua y despliegue

La infraestructura considera mecanismos de integración y despliegue continuo (CI/CD) mediante GitHub Actions.

Este proceso permitirá:

- Automatizar pruebas
- Validar builds
- Generar despliegues controlados
- Reducir errores manuales

Flujo CI/CD

1. Push al repositorio GitHub.
2. Ejecución automática de pipeline.
3. Validación y pruebas.

4. Construcción de contenedores Docker.
5. Despliegue hacia entornos AWS.
6. Actualización controlada de servicios.

3.6.6 Seguridad de Infraestructura

La infraestructura considera mecanismos de seguridad orientados a proteger:

- Acceso
- Comunicaciones
- Servicios
- Persistencia de datos

Controles implementados

Control	Función
HTTPS/TLS	Cifrado de comunicaciones
JWT	Autenticación segura
Cloudflare	Proteccion perimetral
IAM	Gestión de permisos cloud
Roles y permisos	Restriccion de acceso
Backups automáticos	Proteccion de información
Logs y auditoria	Trazabilidad

3.6.7 Descripción general de infraestructura

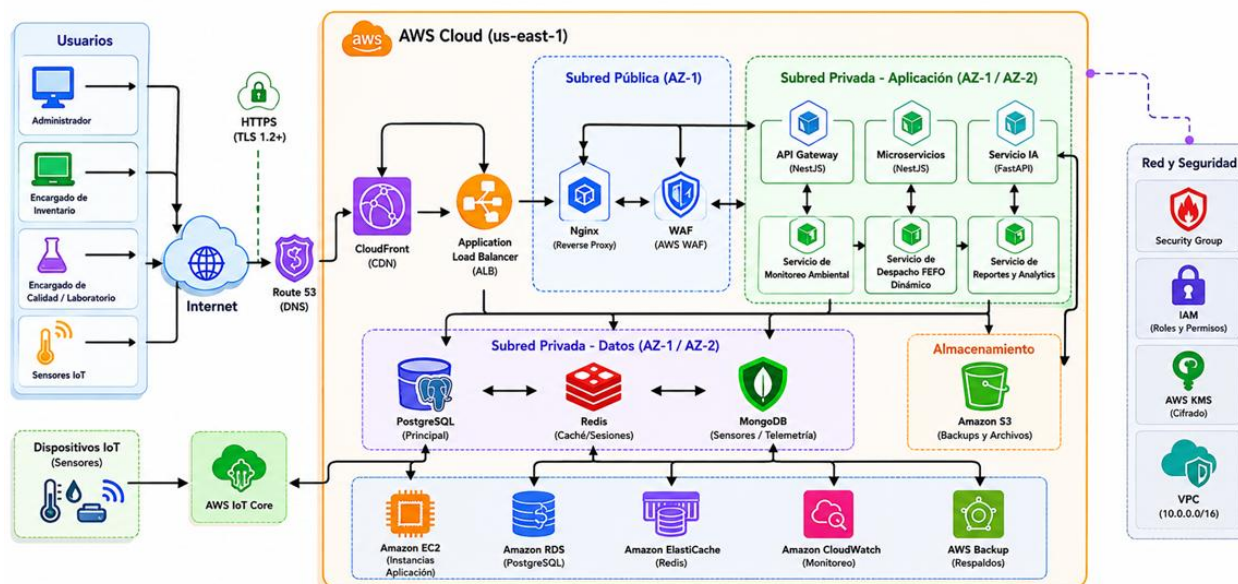
La infraestructura propuesta para G.A.B.I. permite soportar una arquitectura moderna, modular y escalable, capaz de integrar monitoreo ambiental, procesamiento analítico, trazabilidad y comunicación en tiempo real.

La utilización de Docker, AWS, Redis y PostgreSQL permite desacoplar componentes críticos de la solución y facilitar futuras expansiones del sistema, incluyendo integración con nuevos sensores IoT, servicios analíticos adicionales y escalamiento horizontal de los servicios backend.

Además, la incorporación de mecanismos CI/CD, autenticación segura y monitoreo continuo fortalece la mantenibilidad, disponibilidad y estabilidad operacional de la plataforma.

DIAGRAMA DE INFRAESTRUCTURA

Sistema G.A.B.I. – Gestión Analítica de Bio-Insumos



3.7 Diagrama de Arquitectura

El diagrama de arquitectura representa la visión global de la plataforma G.A.B.I., integrando las capas funcionales, componentes tecnológicos, servicios de infraestructura y mecanismos de comunicación que permiten el funcionamiento de la solución.

La arquitectura propuesta se basa en un modelo web modular y distribuido, orientado a separar responsabilidades entre presentación, lógica de negocio, procesamiento analítico, persistencia de datos, seguridad y comunicación en tiempo real.

3.7.1 Capas de la arquitectura

Capa	Componente	Función principal
Presentación	React + TypeScript	Interfaz web, dashboards, formularios y visualización
Seguridad	JWT + HTTPS/TLS	Autenticación, autorización y cifrado
Negocio	NestJS	Inventario, lotes, FEFO dinámico, alertas y auditoría

Tiempo real	WebSockets + Redis Pub/Sub	Notificaciones, eventos y alertas en tiempo real
Analítica / IA	Python + FastAPI	Predicción de demanda y análisis de riesgo
Datos	PostgreSQL	Persistencia relacional y trazabilidad
Infraestructura	Docker + AWS	Despliegue, operación y escalabilidad
Integración continua	GitHub Actions	Automatización de pruebas y despliegue

3.7.2 Descripción de la arquitectura

La arquitectura de G.A.B.I. se organiza en capas con el propósito de mejorar la mantenibilidad, escalabilidad y seguridad de la solución. La capa de presentación, implementada con React y TypeScript, permite a los usuarios interactuar con el sistema mediante una interfaz web responsiva, orientada a dashboards, formularios de inventario, monitoreo ambiental y visualización de alertas.

La capa de negocio está desarrollada en NestJS, actuando como núcleo central de la solución. Este componente administra la lógica de inventario, registro de lotes, movimientos, reglas FEFO dinámicas, autenticación, autorización, auditoría y generación de alertas.

El procesamiento analítico se delega a un microservicio independiente desarrollado en Python/FastAPI, encargado de ejecutar predicciones de demanda, análisis de riesgo de quiebre de stock y apoyo a la priorización dinámica de lotes.

La persistencia de datos se resuelve mediante PostgreSQL, garantizando consistencia transaccional, integridad referencial y trazabilidad de información crítica. Redis se incorpora como componente de soporte para eventos, caché y comunicación Pub/Sub, permitiendo distribuir alertas en tiempo real hacia los usuarios conectados mediante WebSockets.

3.7.3 Flujo general de la arquitectura

Paso	Descripción
1	El usuario accede a la plataforma desde navegador o dispositivo móvil
2	El frontend React consume APIs REST protegidas mediante JWT

3	NestJS valida permisos y ejecuta reglas de negocio
4	PostgreSQL almacena información transaccional y registros de auditoría
5	Redis distribuye eventos y alertas en tiempo real
6	FastAPI procesa predicciones y análisis inteligentes
7	WebSockets notifican eventos críticos al frontend
8	GitHub Actions automatiza despliegues hacia infraestructura AWS

3.7.4 Mecanismos de seguridad arquitectónica

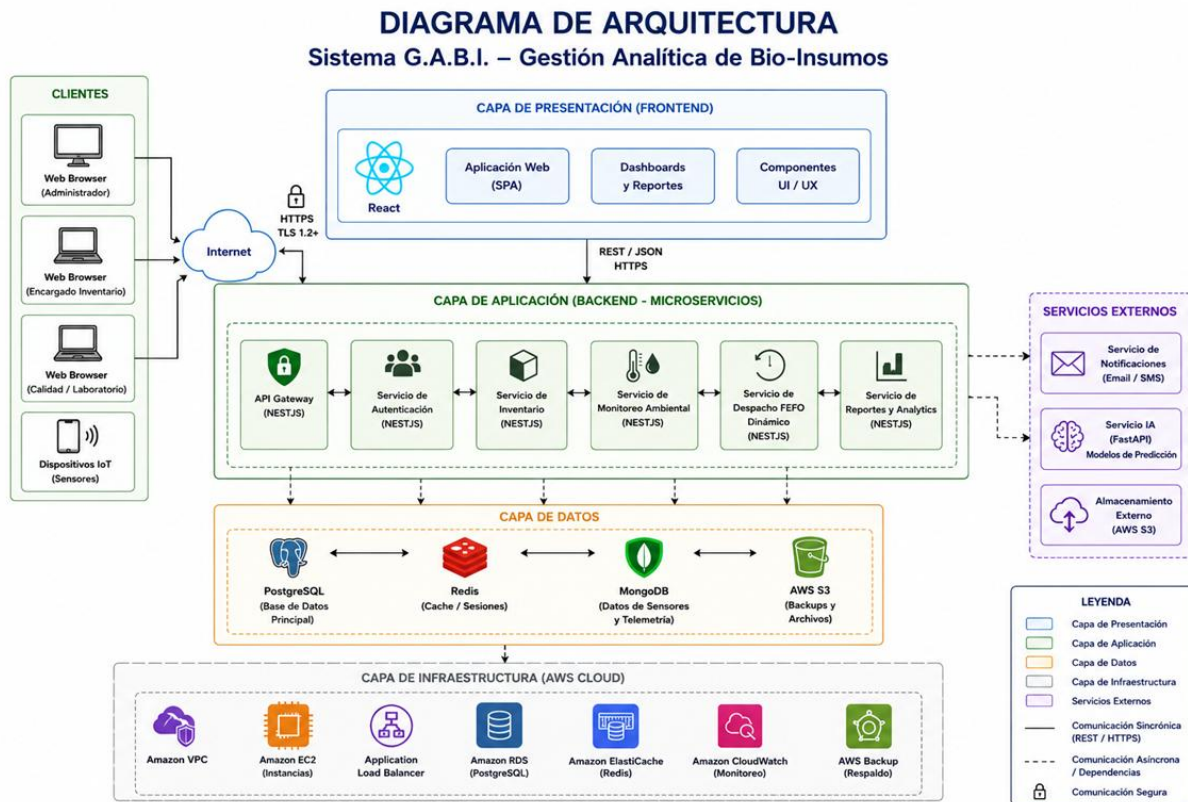
Mecanismo	Aplicación
JWT	Autenticación y autorización de usuarios
HTTPS/TLS	Cifrado de comunicaciones
RBAC	Control de acceso basado en roles
Validación API	Protección de endpoints
Backups	Recuperación de información
Cloudflare	Protección perimetral

3.7.5 Descripción general del diagrama

El diagrama de arquitectura debe representar visualmente la interacción entre los usuarios, la interfaz React, el backend NestJS, el microservicio FastAPI, PostgreSQL, Redis, sensores IoT y servicios de infraestructura AWS.

La arquitectura propuesta permite separar responsabilidades, facilitar la escalabilidad modular y mantener trazabilidad completa de las operaciones. Además, incorpora mecanismos de comunicación en tiempo real, analítica predictiva y seguridad transversal, alineándose con los requerimientos técnicos y operacionales de la plataforma G.A.B.I.

Con esta estructura, la solución puede evolucionar progresivamente hacia un entorno productivo, incorporando nuevos módulos, sensores, servicios analíticos y mecanismos de monitoreo sin afectar la estabilidad general del sistema.



Implementación de los KPI y SLA

Los Indicadores Clave de Desempeño (KPI) permiten medir el comportamiento operacional, técnico y analítico de la plataforma G.A.B.I., asegurando que la solución cumpla con los objetivos de disponibilidad, trazabilidad, monitoreo ambiental y eficiencia en la gestión de bio-insumos críticos.

Los KPI definidos consideran métricas asociadas a disponibilidad de servicios, desempeño de la inteligencia artificial, estabilidad de comunicaciones, monitoreo en tiempo real y control operacional de inventario.

4.1.1 KPI de disponibilidad de la plataforma

Campo	Descripción
Nombre del KPI	Disponibilidad de la plataforma
Objetivo	Garantizar continuidad operacional del sistema
Formula	$(\text{Horas activas} / \text{Horas totales}) \times 100$

Meta	≥ 99.5% mensual
Frecuencia de medición	Mensual
Responsable	Administrador TI
Tipo de KPI	Operacional
Fuente de datos	Logs del sistema y monitoreo
Herramientas asociadas	AWS CloudWatch / Docker Logs

Característica SMART

Elemento SMART	Descripción
Specific	Medir disponibilidad del sistema
Measurable	Porcentaje de Uptime
Achievable	Infraestructura cloud redundante
Relevant	Plataforma crítica para operación
Time-bound	Evaluación mensual

Justificación

Este KPI permite medir la continuidad operacional de la plataforma G.A.B.I., asegurando que los usuarios puedan acceder al sistema de manera estable y permanente. Debido a que la plataforma administra procesos críticos relacionados con bio-insumos, la disponibilidad representa un factor fundamental para la operación del laboratorio.

4.1.2 KPI de precisión del modelo de inteligencia artificial

Campo	Descripción
Nombre del KPI	Precisión del modelo de IA
Objetivo	Medir exactitud de las predicciones analíticas
Fórmula	$(\text{Predicciones correctas} / \text{Total de predicciones}) \times 100$
Meta	≥ 92%
Frecuencia de medición	Semanal
Responsable	Administrador analítico / IA
Tipo de KPI	Analítico
Fuente de datos	Resultados del módulo FastAPI
Herramientas asociadas	Python / FastAPI

Característica SMART

Elemento SMART	Descripción
Specific	Evaluar exactitud predictiva
Measurable	Porcentaje de aciertos

Achievable	Modelos entrenados con datos históricos
Relevant	Apoyo a FEFO dinámico
Time-bound	Evaluación semanal

Justificación

El módulo de inteligencia artificial cumple un rol fundamental en el análisis predictivo de demanda y priorización dinámica de bio-insumos. Este KPI permite validar cuantitativamente la efectividad del modelo analítico y asegurar que las recomendaciones generadas mantengan un nivel adecuado de confiabilidad operacional.

4.1.3 KPI de pérdida de paquetes IoT

Campo	Descripción
Nombre del KPI	Tasa de pérdida de paquetes IoT
Objetivo	Medir estabilidad de transmisión de telemetría
Fórmula	$(\text{Paquetes perdidos} / \text{Paquetes enviados}) \times 100$
Meta	$\leq 1\%$ semanal
Frecuencia de medición	Semanal
Responsable	Administrador de infraestructura
Tipo de KPI	Infraestructura
Fuente de datos	Logs de comunicación IoT
Herramientas asociadas	Redis / Backend NestJS

Característica SMART

Elemento SMART	Descripción
Specific	Medir estabilidad de telemetría
Measurable	Porcentaje de pérdida
Achievable	Uso de protocolos confiables
Relevant	Monitoreo ambiental crítico
Time-bound	Evaluación semanal

Justificación

La plataforma depende de información ambiental obtenida mediante sensores IoT. Este KPI permite asegurar que los datos críticos de temperatura y humedad sean transmitidos correctamente hacia el sistema, evitando pérdida de información relevante para la conservación de bio-insumos sensibles.

4.1.4 KPI de tiempo de respuesta de alertas

Campo	Descripción
Nombre del KPI	Tiempo de respuesta de alertas
Objetivo	Medir rapidez de generación de alertas críticas
Fórmula	Tiempo promedio entre detección y notificación
Meta	≤ 5 segundos
Frecuencia de medición	Diario
Responsable	Administrador TI
Tipo de KPI	Operacional
Fuente de datos	Logs WebSocket y Redis

Herramientas asociadas	Redis / WebSockets
------------------------	--------------------

Característica SMART

Elemento SMART	Descripción
Specific	Medir velocidad de alertas
Measurable	Tiempo en segundos
Achievable	Arquitectura en tiempo real
Relevant	Monitoreo crítico
Time-bound	Evaluación diaria

Justificación

La generación oportuna de alertas resulta fundamental para prevenir riesgos asociados a temperatura, humedad o vencimiento de bio-insumos. Este KPI permite medir la capacidad de reacción del sistema ante eventos críticos detectados en tiempo real.

4.1.5 KPI de precisión de inventario

Campo	Descripción
Nombre del KPI	Precisión de inventario
Objetivo	Validar consistencia entre inventario físico y sistema
Fórmula	$(\text{Registros correctos} / \text{Total inventario auditado}) \times 100$
Meta	$\geq 98\%$
Frecuencia de medición	Mensual
Responsable	Jefe de laboratorio
Tipo de KPI	Operacional
Fuente de datos	Auditorías de inventario
Herramientas asociadas	PostgreSQL / Dashboard

Característica SMART

Elemento SMART	Descripción
Specific	Medir exactitud de inventario
Measurable	Porcentaje de coincidencia
Achievable	Uso de trazabilidad FEFO
Relevant	Gestión crítica de bio-insumos
Time-bound	Evaluación mensual

Justificación

La precisión del inventario permite asegurar confiabilidad operacional, trazabilidad y control adecuado de lotes críticos. Este indicador permite detectar inconsistencias, pérdidas o desviaciones dentro de la gestión de bio-insumos administrados por la plataforma.

4.2 Descripción de SLA

Los Acuerdos de Nivel de Servicio (SLA) permiten definir los niveles mínimos de disponibilidad, rendimiento y continuidad operacional que deberá cumplir la plataforma G.A.B.I., asegurando un funcionamiento estable y confiable para la gestión de bio-insumos críticos dentro del laboratorio.

Los SLA definidos consideran disponibilidad de servicios, tiempos de respuesta, sincronización de telemetría, recuperación ante incidentes y continuidad operacional, alineándose con los requerimientos de monitoreo en tiempo real, trazabilidad y procesamiento analítico establecidos para la solución.

ID	Parámetro de Servicio	Objetivo de Nivel de Servicio (SLA)
SLA-01	Tiempo de respuesta API	El 95% de las peticiones REST y consultas analíticas deberán resolverse en un tiempo menor o igual a 400 ms.
SLA-02	Disponibilidad de la plataforma	Garantizar una disponibilidad operacional mensual igual o superior al 99.5%, excluyendo mantenimientos programados.
SLA-03	Tiempo de respuesta ante incidentes críticos	Asignación de personal técnico en un tiempo menor o igual a 15 minutos desde reportada la falla
SLA-04	Tiempo de resolución de incidentes críticos	Resolución o mitigación operacional de incidentes críticos en un tiempo menor o igual a 4 horas
SLA-05	Sincronización de telemetría IoT	El retraso máximo permitido para actualización de telemetría y dashboards será menor o igual a 5 segundos

Continuidad y recuperación operacional

Parámetro	Objetivo
RTO (Recovery Time Objective)	≤ 1 hora
RPO (Recovery Point Objective)	≤ 15 minutos
Ventana de mantenimiento	Domingo de 02:00 a 04:00 hrs
Frecuencia de respaldos	Backups automáticos diarios

Estrategia de recuperación	Restauración cloud mediante contenedores Docker
-----------------------------------	---

Justificación

La definición de SLA permite asegurar que la plataforma responda de manera eficiente frente a los requerimientos críticos asociados a la gestión de bio-insumos, monitoreo ambiental y trazabilidad operacional.

La baja latencia de la API y la sincronización casi en tiempo real permiten reducir retrasos en la visualización de alertas y eventos críticos asociados a temperatura, humedad o vencimiento de lotes. Paralelamente, los mecanismos de disponibilidad y continuidad operacional permiten minimizar riesgos de interrupción del servicio y pérdida de información crítica del laboratorio.

La incorporación de infraestructura cloud, monitoreo continuo y mecanismos de recuperación rápida fortalece la estabilidad general de la solución y su capacidad de respuesta ante incidentes técnicos u operacionales.

Plan de pruebas

El plan de pruebas de la plataforma G.A.B.I. tiene como objetivo asegurar la calidad, estabilidad y confiabilidad de los distintos componentes que conforman la solución, considerando tanto la capa frontend como backend, el módulo de inteligencia artificial, la persistencia de datos y los mecanismos de comunicación en tiempo real.

La estrategia de pruebas se basa en el enfoque de Pirámide de Pruebas propuesto por Mike Cohn, complementado con lineamientos del ISTQB, permitiendo validar progresivamente el comportamiento del sistema desde pruebas unitarias hasta pruebas integrales de usuario y rendimiento.

ID	Tipo de prueba	Objetivo técnico	Herramientas asociadas
PR-01	Pruebas unitarias	Validar funciones, servicios y lógica de negocio de forma aislada	Jest, PyTest
PR-02	Pruebas de integración	Verificar comunicación entre módulos, APIs y base de datos	Supertest, Postman
PR-03	Pruebas API y contratos	Validar estructura de payloads REST y WebSockets	Swagger/OpenAPI, Pact
PR-04	Pruebas End-to-End (E2E)	Simular flujos completos de usuario dentro de la plataforma	Cypress o Playwright
PR-05	Pruebas de carga y rendimiento	Evaluar comportamiento bajo alta concurrencia y telemetría masiva	Apache JMeter o Artillery

5.1.1 Pruebas unitarias

Campo	Descripción
Objetivo	Validar componentes individuales de software
Cobertura esperada	≥ 80%
Componentes evaluados	Servicios NestJS, controladores, lógica FEFO y algoritmos IA
Herramientas	Jest y PyTest
Automatización	Integración mediante pipelines CI/CD

Justificación

Las pruebas unitarias permiten detectar errores de lógica de manera temprana, validando de forma aislada el comportamiento de servicios críticos relacionados con inventario, autenticación, FEFO dinámico y procesamiento analítico.

5.1.2 Pruebas de integración

Campo	Descripción
Objetivo	Verificar interacción entre módulos y persistencia
Componentes evaluados	APIs REST, PostgreSQL, Redis y FastAPI
Herramientas	Supertest, Postman/Newman
Tipo de validación	Integración funcional y transaccional

Justificación

Las pruebas de integración permiten validar la correcta comunicación entre frontend, backend, base de datos y microservicios, asegurando consistencia de datos y estabilidad operacional.

5.1.3 Pruebas API y contratos

Campo	Descripción
Objetivo	Validar estructura y consistencia de payloads
Componentes evaluados	Endpoints REST y eventos WebSocket

Herramientas	Swagger/OpenAPI y Pact
Tipo de validación	Contratos JSON y estructura de respuestas

Justificación

Estas pruebas permiten asegurar que la comunicación entre frontend React, backend NestJS y servicios externos mantenga formatos consistentes y compatibles, evitando errores de integración entre componentes distribuidos.

5.1.4 Pruebas End-to-End (E2E)

Campo	Descripción
Objetivo	Simular comportamiento real de usuarios
Flujos evaluados	Registro, despacho FEFO, alertas y dashboards
Herramientas	Cypress o Playwright
Tipo de validación	Flujo completo de operación

Justificación

Las pruebas E2E permiten validar la experiencia completa de usuario dentro de la plataforma, asegurando que los distintos módulos operen correctamente de extremo a extremo bajo escenarios reales de laboratorio.

5.1.5 Pruebas de carga y rendimiento

Campo	Descripción
Objetivo	Evaluar estabilidad bajo alta concurrencia
Escenario evaluado	Múltiples sensores IoT y usuarios concurrentes
Herramientas	Apache JMeter o Artillery
Métrica principal	Latencia y capacidad de respuesta

Justificación

La plataforma debe ser capaz de soportar múltiples eventos simultáneos asociados a telemetría, consultas de dashboard y generación de alertas. Estas pruebas permiten detectar cuellos de botella y validar la capacidad operacional de la infraestructura.

5.1.6 Automatización y aseguramiento de calidad

La plataforma incorporará mecanismos de automatización de pruebas mediante pipelines CI/CD utilizando GitHub Actions, permitiendo ejecutar validaciones automáticas durante el ciclo de desarrollo y despliegue. Este enfoque permitirá:

- Detectar errores tempranamente

- Mejorar la estabilidad del software
- Reducir fallos en producción
- Mantener control sobre la calidad del código.

5.1.7 Descripción general del plan de pruebas

La estrategia de pruebas propuesta para G.A.B.I. permite validar progresivamente todos los componentes críticos de la solución, incluyendo frontend, backend, inteligencia artificial, persistencia de datos y comunicación en tiempo real.

La combinación de pruebas unitarias, integración, contratos, E2E y rendimiento permite asegurar que la plataforma mantenga estabilidad, escalabilidad y confiabilidad operacional, alineándose con los requerimientos de monitoreo, trazabilidad y control de bio-insumos críticos definidos para el proyecto.

5.2 Normas y Estándares

La plataforma G.A.B.I. incorporará normas, estándares y buenas prácticas orientadas a asegurar la calidad del software, la seguridad de la información, la correcta definición de requerimientos, la protección de aplicaciones web y la mantenibilidad de la solución.

ID	Norma / Estándar	Aplicación en el proyecto
NE-01	ISO/IEC 25010	Evaluación de calidad del producto software, considerando funcionalidad, rendimiento, usabilidad, fiabilidad, seguridad y mantenibilidad
NE-02	ISO/IEC 27001	Gestión de seguridad de la información, protección de datos, control de accesos, trazabilidad y resguardo de información crítica
NE-03	ISO/IEC/IEEE 29148	Definición, documentación y validación de requerimientos funcionales y no funcionales
NE-04	OWASP Top 10	Identificación y mitigación de riesgos de seguridad en aplicaciones web y APIs REST
NE-05	OpenAPI 3.0	Documentación estandarizada de endpoints REST para pruebas e integración
NE-06	RESTful API	Diseño de servicios web estructurados, escalables y mantenibles
NE-07	GitFlow / Control de versiones	Organización del desarrollo colaborativo mediante ramas, versionamiento y control de cambios

Justificación

La incorporación de normas y estándares permite fortalecer la calidad, seguridad y mantenibilidad de la plataforma G.A.B.I. ISO/IEC 25010 permite evaluar la calidad del software desde atributos medibles, mientras que ISO/IEC 27001 entrega lineamientos para proteger la confidencialidad, integridad y disponibilidad de la información crítica asociada a bio-insumos, lotes, usuarios y registros de auditoría.

Asimismo, ISO/IEC/IEEE 29148 permite estructurar correctamente los requerimientos del sistema, evitando ambigüedades durante el desarrollo. OWASP Top 10 refuerza la seguridad de la aplicación frente a vulnerabilidades comunes en APIs y sistemas web, mientras que OpenAPI 3.0 y RESTful API permiten estandarizar la comunicación entre frontend, backend y servicios externos.

Finalmente, el uso de GitFlow y control de versiones facilita el trabajo colaborativo del equipo, permitiendo gestionar cambios, revisiones y despliegues de manera ordenada y trazable.

Gestión de Disponibilidad

La gestión de disponibilidad de la plataforma G.A.B.I. tiene como objetivo asegurar la continuidad operacional de los servicios críticos asociados al monitoreo ambiental, gestión de bio-insumos, trazabilidad y procesamiento analítico.

La estrategia propuesta considera infraestructura cloud, redundancia de servicios, monitoreo proactivo y mecanismos de recuperación rápida, permitiendo reducir interrupciones y garantizar estabilidad operacional frente a incidentes técnicos.

Estrategias de disponibilidad

ID	Estrategia	Descripción técnica
GD-01	Arquitectura de alta disponibilidad	Despliegue distribuido de servicios backend y microservicios en infraestructura cloud redundante
GD-02	Redundancia de datos y caché	Uso de PostgreSQL y Redis para persistencia, réplica y optimización de consultas
GD-03	Monitoreo proactivo	Supervisión continua de CPU, memoria, latencia y estado de servicios
GD-04	Recuperación automática de servicios	Reinicio controlado de contenedores Docker ante fallos
GD-05	Comunicación en tiempo real	Uso de Redis Pub/Sub y WebSockets para distribución estable de eventos

6.1.1 Arquitectura de alta disponibilidad

La solución considera una arquitectura cloud orientada a reducir puntos únicos de fallo mediante la separación de servicios y contenedores independientes.

El backend NestJS, frontend React y microservicios FastAPI podrán ejecutarse de forma desacoplada utilizando Docker, facilitando recuperación rápida, mantenibilidad y escalabilidad progresiva.

6.1.2 Redundancia y persistencia de datos

La plataforma utilizará PostgreSQL como motor principal de persistencia transaccional y Redis como mecanismo de caché y distribución de eventos.

La estrategia considera:

- Respallos automáticos

- Almacenamiento persistente
- Recuperación rápida de información crítica asociada a inventario, auditoría y telemetría.

6.1.3 Monitoreo proactivo

La infraestructura incorporará mecanismos de monitoreo continuo para detectar degradaciones de rendimiento antes de que afecten la operación del sistema.

Las métricas monitoreadas incluyen:

- Uso de CPU
- Memoria RAM
- Latencia
- Disponibilidad API
- Conectividad IoT
- Estado de contenedores

6.1.4 Recuperación automática de servicios

La utilización de Docker permitirá reiniciar servicios afectados automáticamente ante fallos operacionales o errores de ejecución, reduciendo tiempos de interrupción y facilitando continuidad operacional.

6.1.5 Justificación

La disponibilidad continua representa un aspecto crítico para la operación de G.A.B.I., debido a que la plataforma administra información sensible relacionada con bio-insumos, monitoreo ambiental y trazabilidad operacional.

La incorporación de infraestructura cloud, monitoreo proactivo, contenedorización y redundancia de servicios permite mejorar la estabilidad de la solución y reducir riesgos asociados a fallas técnicas o pérdida de disponibilidad del sistema.

6.2 Gestión de continuidad

La gestión de continuidad de la plataforma G.A.B.I. tiene como objetivo asegurar la recuperación y continuidad operacional del sistema frente a incidentes críticos, fallas de infraestructura, pérdida de conectividad o eventos que puedan comprometer la disponibilidad de los servicios.

La estrategia considera mecanismos de respaldo, recuperación, redundancia y tolerancia a fallos, permitiendo minimizar tiempos de interrupción y proteger la integridad de la información asociada a bio-insumos, inventario y monitoreo ambiental.

Estrategia de continuidad operacional

ID	Estrategia	Descripción técnica
GC-01	Plan de recuperación ante desastres (DRP)	Respaldo y recuperación automatizada de información crítica

GC-02	Estrategia de failover	Redirección de servicios ante fallos de infraestructura
GC-03	Continuidad de telemetría IoT	Almacenamiento temporal de datos ante pérdida de conectividad
GC-04	Backups automáticos	Generación periódica de respaldos de base de datos
GC-05	Recuperación mediante contenedores	Restauración rápida de servicios Docker

6.2.1 Plan de recuperación ante desastres

La plataforma incorporará mecanismos automatizados de respaldo y recuperación para proteger la información almacenada en PostgreSQL y los registros asociados a auditoría, inventario y telemetría.

La estrategia considera:

- Backups automáticos diarios
- Almacenamiento cloud
- Restauración controlada
- Recuperación rápida de servicios críticos

6.2.2 Estrategia de failover

La infraestructura permitirá redirigir servicios críticos hacia entornos alternativos en caso de falla parcial o indisponibilidad del entorno principal.

El objetivo de esta estrategia es reducir interrupciones operacionales y mantener accesibilidad a la plataforma durante incidentes técnicos relevantes.

6.2.3 Continuidad de telemetría IoT

En caso de pérdida temporal de conectividad, los sensores IoT o dispositivos de captura podrán almacenar temporalmente información ambiental antes de retransmitirla nuevamente hacia la plataforma.

Esta estrategia permite evitar pérdida de datos críticos relacionados con temperatura, humedad y monitoreo ambiental.

6.2.4 Backups y recuperación de datos

Parámetro	Objetivo
Frecuencia de backups	Diario
Tipo de respaldo	Incremental y completo

Medio de almacenamiento	Cloud Storage
Tiempo máximo de recuperación (RTO)	≤ 1 hora
Punto máximo de pérdida de datos (RPO)	≤ 15 minutos

6.2.5 Recuperación mediante contenedores

La utilización de Docker permitirá restaurar rápidamente los distintos servicios que conforman la plataforma, incluyendo frontend, backend, Redis y microservicios analíticos.

Esto permitirá:

- Reducir tiempos de recuperación
- Facilitar despliegues
- Mejorar continuidad operacional frente a incidentes

6.2.6 Justificación

La continuidad operacional representa un aspecto fundamental para G.A.B.I., debido a que la plataforma administra procesos críticos asociados a monitoreo ambiental, trazabilidad y control de bio-insumos sensibles.

La incorporación de mecanismos de respaldo, failover, recuperación rápida y tolerancia a fallos permite reducir riesgos operacionales y proteger la disponibilidad e integridad de la información frente a eventos críticos o interrupciones de infraestructura.

6.3 Plan de mantención

El plan de mantención de la plataforma G.A.B.I. tiene como objetivo asegurar la estabilidad, seguridad, rendimiento y evolución continua de la solución, considerando actividades preventivas, correctivas y evolutivas sobre los distintos componentes tecnológicos del sistema.

La estrategia contempla mantenimiento periódico sobre infraestructura, base de datos, servicios backend, frontend, microservicios analíticos y mecanismos de monitoreo, permitiendo reducir riesgos de obsolescencia tecnológica y degradación operacional.

ID	Tipo de mantención	Descripción técnica	Frecuencia
MN-01	Mantenimiento preventivo	Actualización de librerías, optimización de base de datos y revisión de infraestructura	Bimestral / Semestral
MN-02	Mantenimiento correctivo	Corrección de errores, incidentes y fallas reportadas	Bajo demanda

MN-03	Mantenimiento evolutivo	Incorporación de mejoras funcionales y optimización de IA	Trimestral
MN-04	Mantenimiento de seguridad	Aplicación de parches y revisión de vulnerabilidades	Mensual
MN-05	Optimización de rendimiento	Ajustes de consultas, caché y procesamiento analítico	Trimestral

6.3.1 Mantenimiento preventivo

El mantenimiento preventivo considera actividades orientadas a evitar degradaciones futuras del sistema mediante revisiones periódicas de software, infraestructura y persistencia de datos.

Las principales actividades incluyen:

- Actualización de dependencias Node.js y Python
- Revisión de vulnerabilidades
- Optimización de índices PostgreSQL
- Revisar de logs
- Monitoreo de consumo de recursos.

6.3.2 Mantenimiento correctivo

El mantenimiento correctivo permitirá resolver incidentes o errores detectados durante la operación de la plataforma.

Las correcciones podrán incluir:

- Solución de bugs
- Ajustes de integración
- Recuperación de servicios
- Corrección de errores de telemetría
- Estabilización de módulos afectados.

Dependiendo de la criticidad, las correcciones podrán desplegarse mediante hotfixes controlas utilizando control de versiones Git.

6.3.3 Mantenimiento evolutivo

El mantenimiento evolutivo permitirá incorporar mejoras funcionales y optimizaciones progresivas dentro de la plataforma G.A.B.I.

Estas mejoras podrán considerar:

- Nuevas funcionalidades,
- Optimización de dashboards,
- Integración de sensores adicionales,

- Mejoras en experiencia de usuario,
- y reajuste de modelos predictivos implementados en FastAPI.

6.3.4 Mantenimiento de seguridad

La plataforma incorporará revisiones periódicas de seguridad orientadas a:

- actualización de librerías vulnerables,
- revisión de autenticación JWT,
- validación de endpoints REST,
- revisión de configuraciones cloud,
- y análisis de riesgos asociados a aplicaciones web.

6.3.5 Optimización de rendimiento

Las actividades de optimización permitirán mantener estabilidad operacional y tiempos de respuesta adecuados frente al crecimiento progresivo de datos y eventos.

Las principales acciones consideran:

- optimización de consultas SQL,
- mejora de índices,
- optimización de Redis,
- ajuste de procesamiento analítico,
- y revisión de rendimiento de contenedores Docker.

6.3.6 Justificación

La mantención continua representa un aspecto fundamental para asegurar la estabilidad y evolución tecnológica de la plataforma G.A.B.I.

La incorporación de actividades preventivas, correctivas, evolutivas y de seguridad permite reducir riesgos operacionales, mejorar la calidad del software y asegurar que la solución mantenga niveles adecuados de rendimiento, disponibilidad y protección de la información a lo largo del tiempo.

Revisión de Carta Gantt y fases del proyecto

La planificación del proyecto G.A.B.I. se estructuró mediante fases progresivas de análisis, diseño, desarrollo, pruebas y despliegue, permitiendo organizar las actividades técnicas y administrativas necesarias para la construcción de la solución.

La estrategia considera una secuencia controlada de actividades orientadas al levantamiento de requerimientos, diseño arquitectónico, implementación de componentes, aseguramiento de calidad y puesta en marcha de la plataforma.

Fases principales del proyecto

Fase	Objetivo principal	Entregables esperados
Fase 1: Análisis e iniciación	Definir alcance, requerimientos y factibilidad	Documento de requerimientos y análisis técnico

Fase 2: Diseño de arquitectura	Modelar arquitectura, datos e infraestructura	Diagramas UML, BPMN, arquitectura y modelo de datos
Fase 3: Construcción e integración	Desarrollar frontend, backend, IA e integración IoT	Plataforma funcional integrada
Fase 4: QA y despliegue	Ejecutar pruebas, despliegue y estabilización	Sistema validado y documentación final

7.1.1 Fase 1: Análisis e iniciación

Actividad	Descripción
Definición de alcance	Delimitación funcional y técnica del proyecto
Levantamiento de requerimientos	Identificación de requerimientos funcionales y no funcionales
Evaluación de factibilidad	Análisis técnico y económico
Planificación inicial	Organización de tareas y recursos
Validación inicial	Revisión y aprobación conceptual

7.1.2 Fase 2: Diseño de arquitectura

Actividad	Descripción
Diseño base de datos	Modelamiento lógico y relacional
Diseño UI/UX	Wireframes y dashboards
Diseño arquitectónico	Diagramas de componentes, infraestructura y comunicaciones
Definición de APIs	Modelado de endpoints y payloads
Validación arquitectónica	Revisión técnica y análisis de riesgos

7.1.3 Fase 3: Construcción e integración

Actividad	Descripción
Desarrollo backend	Implementación NestJS y lógica FEFO
Desarrollo frontend	Construcción de interfaz React
Desarrollo IA	Implementación FastAPI y predicción
Integración IoT	Captura y procesamiento de telemetría
Integración general	Comunicación entre módulos

7.1.4 Fase 4: QA y despliegue

Actividad	Descripción
Pruebas y QA	Validación funcional y rendimiento
Despliegue cloud	Configuración AWS y Docker
Seguridad y HTTPS	Configuración TLS/JWT
Capacitación	Marcha blanca y entrenamiento
Cierre del proyecto	Entrega de documentación y validación final

7.1.5 Justificación

La planificación propuesta permite organizar progresivamente las distintas etapas técnicas y operacionales del proyecto G.A.B.I., asegurando control sobre el desarrollo, integración y validación de la solución.

La división por fases facilita la trazabilidad de actividades, mejora la gestión de riesgos y permite mantener coherencia entre arquitectura, desarrollo, pruebas y despliegue, alineándose con buenas prácticas de ingeniería de software y gestión de proyectos tecnológicos.

Referencia a anexo

El cronograma detallado del proyecto G.A.B.I., incluyendo distribución temporal de actividades, planificación por fases, entregables técnicos y tareas asociadas al desarrollo de la solución, se encuentra disponible en el Anexo A — Carta Gantt detallada del proyecto. El cronograma detallado del proyecto G.A.B.I., incluyendo distribución temporal de actividades, planificación por fases, entregables técnicos y tareas asociadas al desarrollo de la solución, se encuentra disponible en el Anexo A — Carta Gantt detallada del proyecto.

Carta Gantt



Conclusiones

El desarrollo del proyecto G.A.B.I. — Gestión Analítica de Bio-Insumos — permitió diseñar una solución tecnológica orientada a mejorar la trazabilidad, monitoreo y control operacional de bio-insumos críticos dentro de entornos de laboratorio, integrando herramientas modernas de desarrollo web, procesamiento analítico, monitoreo ambiental e infraestructura cloud.

A lo largo del informe se definieron los requerimientos funcionales y no funcionales de la plataforma, junto con una arquitectura modular basada en React, NestJS, FastAPI, PostgreSQL y Redis, permitiendo construir una solución escalable, desacoplada y preparada para operar en escenarios de monitoreo en tiempo real y procesamiento distribuido.

La incorporación de mecanismos FEFO dinámicos, monitoreo de telemetría IoT, generación automática de alertas y capacidades de inteligencia artificial permiten que la plataforma no solo administre inventario, sino que además apoye la toma de decisiones preventivas y reduzca riesgos asociados al vencimiento, deterioro o pérdida de bio-insumos sensibles.

Asimismo, la definición de KPI, SLA, mecanismos de disponibilidad, continuidad operacional, aseguramiento de calidad y estándares internacionales permitió estructurar una solución alineada con buenas prácticas de ingeniería de software, seguridad de la información y gestión de infraestructura tecnológica.

Desde una perspectiva técnica, el proyecto evidencia la integración de múltiples disciplinas relacionadas con desarrollo de software, arquitectura de sistemas, bases de datos, cloud computing, IoT y analítica predictiva, demostrando la viabilidad de implementar plataformas inteligentes orientadas a entornos críticos y altamente regulados.

Finalmente, la propuesta desarrollada para G.A.B.I. establece una base sólida para futuras extensiones funcionales, incluyendo integración con nuevos sensores, automatización avanzada de procesos de laboratorio, ampliación de modelos predictivos y escalamiento hacia arquitecturas distribuidas de mayor capacidad, consolidando una solución tecnológica moderna, escalable y alineada con los desafíos actuales de la transformación digital en la gestión de bio-insumos.

Referencias bibliográficas

Amazon Web Services (AWS). (2024). *AWS Well-Architected Framework – Reliability Pillar*.

<https://aws.amazon.com/architecture/well-architected/>

Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional.

Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com/>

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.

IEEE. (2016). *IEEE 1012-2016 — Standard for System, Software, and Hardware Verification and Validation*.

International Organization for Standardization (ISO). (2011). *ISO/IEC 25010: Systems and Software Engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*.

International Organization for Standardization (ISO). (2022). *ISO/IEC 27001: Information Security, Cybersecurity and Privacy Protection*.

ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148: Systems and software engineering — Requirements engineering*.

NestJS. (2024). *NestJS Official Documentation*. <https://docs.nestjs.com/>

OpenAPI Initiative. (2024). *OpenAPI Specification Version 3.0*. <https://swagger.io/specification/>

OWASP Foundation. (2021). *OWASP Top 10*. <https://owasp.org/www-project-top-ten/>

PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>

Project Management Institute (PMI). (2021). *PMBOK Guide* (7th ed.).

React Documentation Team. (2024). *React Official Documentation*. <https://react.dev/>

Redis Ltd. (2024). *Redis Documentation*. <https://redis.io/docs/>